



Министерство образования и науки Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Иркутский государственный университет»
(ФГБОУ ВО «ИГУ»)
Факультет бизнес-коммуникаций и информатики

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ
Б2.В.04(П) Технологическая (проектно-технологическая) практика
по направлению 09.03.03 «Прикладная информатика»
(прикладной бакалавриат)
профиль «Прикладная информатика в дизайне»

Интеллектуальная система прогнозирования заполняемости средств размещения
на основе данных туристических агрегаторов и событийной активности в регионе

Студент 4 курса очной формы
обучения, группа 14422-ДБ
_____ В.П. Исполатов

Руководитель: ст.преп.
_____ О.А. Инешина

Работа защищена:
«_____» _____ 2026 г.
С оценкой _____

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
1 ИССЛЕДОВАНИЕ МЕТОДОВ МОНИТОРИНГА И ПРОГНОЗИРОВАНИЯ ЗАПОЛНЯЕМОСТИ СРЕДСТВ РАЗМЕЩЕНИЯ.....	7
1.1 Мониторинг туристической активности: задачи и существующие решения	7
1.2 Обоснование выбора технологий и инструментов	11
1.3 Обоснование выбора методов прогнозирования и фреймворков реализации интеллектуального агента	18
Выводы по главе.....	22
2 РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ ПРОГНОЗИРОВАНИЯ ЗАПОЛНЯЕМОСТИ СРЕДСТВ РАЗМЕЩЕНИЯ.....	23
2.1 Архитектура системы и подсистема сбора данных	23
2.2 Модуль прогнозирования и интеллектуальный агент.....	29
2.3 Проектирование пользовательского интерфейса и обеспечение качества .	35
Выводы по главе.....	46
ЗАКЛЮЧЕНИЕ	47
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	49
ПРИЛОЖЕНИЯ.....	53
Приложение А Скриншоты интерфейса системы	53
Приложение Б Техническая конфигурация инфраструктуры	59
Приложение В Структура API (59 эндпоинтов)	60
Приложение Г Таблица признаков модуля FeatureEngineering.....	61
Приложение Д Ежедневные записи студента по практике.....	64

ВВЕДЕНИЕ

Иркутская область занимает особое место в системе внутреннего туризма России. По данным Росстата, регион ежегодно принимает свыше 1,5 миллиона туристов [1]. Озеро Байкал, включённое в список Всемирного наследия ЮНЕСКО в 1996 году, остаётся главным магнитом турпотока; стратегия развития туризма области до 2030 года предусматривает рост до 3 миллионов человек в год [2]. Гостиничная инфраструктура насчитывает свыше тысячи объектов размещения в 42 муниципальных районах и городских округах. Сезонность выражена резко: Ольхонский район — главный драйвер летнего потока с заполняемостью до 95 % в июле, зимой показатель падает ниже 20 %. Слюдянский район привлекает туристов круглогодично благодаря Кругобайкальской железной дороге, горнолыжному курорту «Гора Соболиная» и зимнему фестивалю «Ледовый Байкал». Подобные перепады создают для средств размещения серьёзную проблему ценообразования и кадрового планирования.

Данные о заполняемости рассредоточены по десяткам агрегаторов бронирования. Каждый из них охватывает лишь часть рынка и отдаёт информацию в собственном формате без открытых программных интерфейсов (API, Application Programming Interface). Нами проведён анализ более 30 российских платформ — Ostrovok, Sutochno, Яндекс.Путешествия, TVIL, Bronevik и других — который выявил единственный сервис с открытым API: 101Hotels возвращает данные в формате JSON (JavaScript Object Notation) без авторизации [3]. Остальные агрегаторы защищены антибот-системами либо требуют партнёрского договора. Классические методы прогнозирования — ARIMA (AutoRegressive Integrated Moving Average), метод Хольта-Уинтерса — не располагают штатными механизмами учёта событийной активности и погодных факторов, что ограничивает их применимость к специфике Байкальского региона.

Проблему усиливает отсутствие визуально-аналитических инструментов. Управляющие компании принимают решения на основе субъективных оценок, а

региональные органы туризма не располагают оперативной картиной заполняемости номерного фонда в масштабе области. Готовые решения, ориентированные именно на Иркутскую область, в ходе проведённого анализа не обнаружены.

Актуальность работы определяется потребностью в интеллектуальной аналитической системе (ИАС), способной автоматически собирать данные из доступных источников, обогащать их событийной и метеорологической информацией, строить ансамблевые прогнозы заполняемости и предоставлять результаты через веб-интерфейс с продуманной информационной архитектурой и диалоговым агентом на основе большой языковой модели (LLM, Large Language Model).

Объект исследования — рынок средств размещения Иркутской области.

Предмет исследования — методы автоматизированного сбора, визуального анализа и прогнозирования заполняемости средств размещения с применением ансамблей моделей машинного обучения, интеллектуальных агентов и средств информационного дизайна.

Цель работы — проектирование и разработка интеллектуальной аналитической системы прогнозирования заполняемости средств размещения с интерфейсом визуальной аналитики на основе данных туристических агрегаторов и событийной активности в регионе.

Для достижения поставленной цели определены следующие **задачи**:

1. Исследовать предметную область мониторинга туристической активности и провести анализ существующих аналитических решений.
2. Обосновать выбор технологий, дизайн-инструментов и средств визуализации данных для разработки системы.
3. Спроектировать и реализовать подсистему автоматизированного сбора данных из туристических агрегаторов и порталов событий.
4. Разработать ансамблевый метод прогнозирования заполняемости средств размещения и модуль его реализующий с визуальной аналитикой результатов.

5. Спроектировать архитектуру интеллектуальной аналитической системы (ИАС) и реализовать систему, ее взаимодействие с интеллектуальным агентом и веб-интерфейс.

Методы исследования: системный анализ предметной области, сравнительный анализ информационных технологий и средств визуализации, объектно-ориентированное проектирование и реализация программных систем, итеративное прототипирование пользовательского интерфейса (UI, User Interface), методы машинного обучения (градиентный бустинг, нейросетевые модели временных рядов, ансамблирование), экспериментальная оценка качества прогнозов на реальных данных.

Научная новизна работы состоит в разработке ансамблевого метода прогнозирования заполняемости средств размещения, объединяющего три модели машинного обучения различной природы (Prophet, NeuralProphet, XGBoost) с адаптивными весами на основе обратной среднеквадратичной ошибки (inverse-RMSE, Root Mean Square Error). Для Иркутской области впервые реализована интеграция данных о событийной активности и метеорологических факторов как экзогенных переменных ансамблевого прогноза. Среднеквадратичная ошибка ансамбля составила 3,85 п.п. для Иркутского района при лучшем одиночном результате 6,43 п.п. Предложен подход к визуализации туристической аналитики, опирающийся на принципы визуальной иерархии и снижения когнитивной нагрузки. Разработана архитектура интеллектуального агента на базе графа состояний LangGraph с пятью инструментами для диалогового доступа к аналитике на естественном языке.

Практическая значимость работы определяется тем, что разработанная ИАС в автоматическом режиме собирает данные о заполняемости объектов размещения Иркутской области, формирует ансамблевые прогнозы на горизонте 3–90 дней и визуализирует результаты на восьми интерактивных страницах: панель мониторинга (дашборд) ключевых показателей эффективности, временные ряды заполняемости, тепловые карты, радарные диаграммы регионов, географическая карта отелей. Веб-интерфейс спроектирован по

принципам UX-дизайна (User Experience, пользовательский опыт): информационная архитектура «от общего к частному», адаптивная вёрстка, тёмная и светлая темы, чат-интерфейс AI-агента с подсказками по модели conversational UI (диалоговый интерфейс). Система адресована управляющим компаниям гостиничной отрасли для оптимизации ценообразования и региональным органам управления туризмом для оперативного мониторинга.

1 ИССЛЕДОВАНИЕ МЕТОДОВ МОНИТОРИНГА И ПРОГНОЗИРОВАНИЯ ЗАПОЛНЯЕМОСТИ СРЕДСТВ РАЗМЕЩЕНИЯ

1.1 Мониторинг туристической активности: задачи и существующие решения

Мониторинг туристической активности представляет собой систематический процесс сбора, обработки и анализа данных о перемещениях и поведении туристов, состоянии инфраструктуры размещения и факторах, влияющих на спрос. Заполняемость (occupancy rate) — процент занятых номеров от общего номерного фонда — выступает центральным показателем гостиничного сектора. Этот индикатор определяет выручку объекта размещения и позволяет оценить спрос и туристическую привлекательность территории на его основе. Помимо заполняемости, задачи мониторинга охватывают отслеживание средней стоимости номера (ADR, Average Daily Rate), дохода на доступный номер (RevPAR, Revenue per Available Room), структуры спроса по каналам бронирования и сезонных колебаний.

Иркутская область — один из ведущих туристических регионов Сибири [4]. Территория охватывает 42 муниципальных района и городских округа, из которых 15 имеют выраженную туристическую специализацию: Иркутский, Ольхонский, Слюдянский, Листвянский (в составе Иркутского района), Байкальский и другие. Географическая неоднородность порождает различные модели спроса. Ольхонский район демонстрирует резкую сезонность с пиками в июле–августе и январе–марте (ледовый сезон Байкала) [4]. Иркутский район с деловым туризмом областного центра характеризуется более равномерной заполняемостью. Слюдянский район привлекает транзитных туристов на маршруте Кругобайкальской железной дороги (КБЖД) и горнолыжном курорте «Гора Соболиная» [4] преимущественно в тёплый сезон и зимний период соответственно.

Сезонные закономерности (паттерны) заполняемости осложняются влиянием экзогенных факторов. Государственные праздники и школьные каникулы формируют предсказуемые всплески: новогодние каникулы (31

декабря–8 января), майские праздники, июньские выпускные. Культурные и спортивные события — фестиваль «Байкальская миля», «Зимниада», международный фестиваль «Звёзды на Байкале» [4] — создают локальные пики, ограниченные конкретными районами и датами. Погода оказывает нетривиальное воздействие: аномально тёплая зима сокращает период ледовых экскурсий, а затяжные дожди летом снижают поток в палаточные лагеря, перераспределяя спрос в пользу капитальных средств размещения. Совокупное влияние перечисленных факторов делает задачу мониторинга многомерной — учёт одного лишь календаря недостаточен для прогнозирования спроса.

Обзор существующих решений для мониторинга гостиничной заполняемости выявляет несколько категорий систем. Корпоративные платформы управления доходами (Revenue Management Systems, RMS) — IDeaS, Duetto, RateGain — ориентированы на крупные гостиничные сети. Они работают с внутренними данными PMS (Property Management System) конкретного объекта, используют алгоритмы динамического ценообразования и оптимизации доходности. Стоимость лицензии начинается от нескольких тысяч долларов в месяц [5]. Для малых средств размещения, составляющих основу гостиничного рынка Прибайкалья, такие затраты несопоставимы с годовой выручкой. Кроме того, RMS не работают с открытыми данными агрегаторов и не учитывают региональный событийный календарь.

Аналитические дашборды агрегаторов бронирования предоставляют ограниченную статистику партнёрам. Airbnb публикует AirDNA [6] — платформу с данными о заполняемости и ценах краткосрочной аренды, ориентированную на рынок P2P-аренды (Peer-to-Peer, одноранговая модель). Booking.com через Analytics Dashboard показывает конверсии и тренды спроса для зарегистрированных объектов. Инструменты охватывают исключительно собственную базу объявлений (листингов), не интегрируют внешние факторы и не предоставляют прогностических возможностей. Для региона с сотнями средств размещения на десятках платформ такая фрагментация означает неполноту картины.

Государственная система «Электронная путёвка» [7] (Единая информационная система электронных путёвок, ЕИСЭП) и ведомственные мониторинги Ростуризма (ныне — Министерства экономического развития) оперируют агрегированными квартальными данными. Оперативной информации о заполняемости в разрезе объектов или районов они не предоставляют. Для принятия решений о ценообразовании, которые требуются ежедневно, такая периодичность неприемлема.

Из аналитических платформ общего назначения применяются Yandex DataLens и Google Looker Studio. DataLens строит визуализации на основе произвольных источников данных и использовался на ранних этапах разработки описываемой системы. Зависимость от облачной инфраструктуры Yandex Cloud и ограниченные возможности кастомизации интерактивных элементов (всплывающие подсказки (тултипы), вложенные фильтры, адаптивность под мобильные устройства) потребовали перехода к встроенным визуализациям на базе React-библиотек [8]. Google Travel Insights предоставляет агрегированную статистику поисковых запросов по направлениям, но не содержит данных о фактической заполняемости и не покрывает российский сегмент в полном объёме после 2022 года.

Ограничения перечисленных решений сводятся к трём проблемам:

- ни одна из существующих платформ не агрегирует данные о заполняемости средств размещения в масштабе региона — каждая работает с собственной, неполной базой;
- прогнозные модули коммерческих RMS не учитывают специфику регионального событийного календаря и метеорологических условий Прибайкалья;
- стоимость корпоративных решений несопоставима с бюджетами малых гостиничных объектов, преобладающих в регионе.

Для оценки доступности данных нами проведён анализ программных интерфейсов российских агрегаторов методом обратной инженерии: перехват HTTP-запросов (HyperText Transfer Protocol, протокол передачи гипертекста)

через инструменты разработчика браузера (F12-подход). Обследованы 30 агрегаторов, для 13 крупнейших выполнена проверка наличия скрытых конечных точек (endpoint-ов) JSON API. Результаты систематизированы в таблице 1.

Таблица 1

Анализ API туристических агрегаторов (извлечение)

Агрегатор	Проверенные endpoint-ы	Результат
101Hotels	/city/{slug}, /region/{slug}	Открытый JSON API, данные о заполняемости
Ostrovok.ru	/api/v2/, /api/site/serp/v3/	404 — закрыто, требуется партнёрство ETG
Sutochno.ru	/api/search, /api/v1/, /graphql	403 — полная блокировка
Яндекс.Путешествия	/api/hotels/searchOffers	SmartCaptcha — антибот-защита
Hotellook	/api/v2/cache, /hotels.json	404 — требуется токен партнёра
TVIL.ru	/api/search, /api/v1/catalog	Нет JSON API — возвращает HTML (HyperText Markup Language)
Bronevik.com	/api/search	SOAP XML, NDA (Non-Disclosure Agreement, соглашение о неразглашении), договор

101Hotels оказался единственным российским агрегатором с открытым JSON API, предоставляющим данные о количестве свободных номеров, ценах, координатах и рейтингах без авторизации. Региональные endpoint-ы возвращают данные по 200+ объектам размещения в 30 населённых пунктах. Остальные платформы требуют партнёрского договора, защищены антибот-системами либо вообще не предоставляют структурированных данных. Обнаружение этого endpoint-а выполнено вручную через анализ вкладки Network в DevTools при навигации по сайту 101Hotels.

Для перекрёстной верификации данных привлечён API сервиса Hotelo, предоставляющий информацию из базы TripAdvisor: 1262 объекта Иркутской области с координатами, рейтингами, типами размещения (25 категорий) и

фотографиями [9]. Сопоставление 101Hotels и Xotelo по координатам (формула Хаверсина) выявило 631 объект с перекрёстным рейтингом TripAdvisor.

Событийная информация доступна через восемь источников:

- irk.ru/afisha — городской портал Иркутска;
- culture38.ru — культурный портал Иркутской области;
- zeroevent.ru — агрегатор мероприятий;
- Яндекс.Афиша — афиша событий;
- kassir.ru — билетный сервис;
- «Культура.РФ» — федеральный портал;
- Telegram-каналы (@visitirkutskregion, @baikalgo) — туристическая тематика;

- база крупных событий — фестивали и спортивные мероприятия.

Агрегатор KudaGo, предоставляющий структурированные данные по городам России через REST (Representational State Transfer) API, при проверке не включал Иркутск в список поддерживаемых городов — источник отклонён. Метеорологические данные предоставляет Open-Meteo API — бесплатный сервис без регистрации, обеспечивающий историческую и прогнозную погоду с почасовым разрешением для произвольных координат [10].

Таким образом, объективная потребность в мониторинге заполняемости средств размещения Иркутской области, отсутствие готовых решений, покрывающих региональную специфику, и наличие доступных источников данных (101Hotels API, восемь порталов событий, Open-Meteo) обуславливают необходимость разработки собственной интеллектуальной аналитической системы. Разрабатываемая система должна агрегировать разнородные источники, учитывать экзогенные факторы (погода, события, праздники) и предоставлять прогнозные оценки через веб-интерфейс, спроектированный с учётом принципов информационного дизайна и визуальной иерархии.

1.2 Обоснование выбора технологий и инструментов

Серверная часть системы отвечает за приём HTTP-запросов, взаимодействие с базой данных, запуск модулей извлечения данных (парсеров),

обучение и применение моделей прогнозирования, координацию работы интеллектуального агента. Три ключевых требования определяют выбор технологий: поддержка асинхронной обработки ввода-вывода (I/O) для параллельного обращения к внешним API, совместимость с экосистемой машинного обучения Python и возможность автоматической генерации документации к API.

Веб-фреймворк (программный каркас, framework). Сравнение трёх Python-фреймворков — FastAPI, Django и Flask — выполнено по критериям, определяющим пригодность для аналитической системы (таблица 2).

Таблица 2

Сравнение веб-фреймворков Python

Критерий	FastAPI	Django	Flask
Асинхронность	Нативная (ASGI, async/await)	Частичная (с Django 4.1, ASGI)	Нет (WSGI, требуется gevent)
Автодокументация API	Swagger UI + ReDoc из коробки	Нет (django-rest-swagger устарел)	Нет (flask-restx отдельно)
Валидация данных	Pydantic (встроена)	DRF (Django REST Framework) Serializers	Вручную или marshmallow
Производительность	~15 000 req/s (uvicorn)	~3 000 req/s (gunicorn)	~5 000 req/s (gunicorn)
Экосистема ML	Прямая интеграция	ORM-centric	Минимальная
Кривая обучения	Средняя	Высокая (шаблон MTV)	Низкая

FastAPI выбран по совокупности факторов. Нативная асинхронность на базе стандарта ASGI (Asynchronous Server Gateway Interface) обрабатывает десятки одновременных запросов к внешним API парсеров и LLM-провайдерам без блокировки. Встроенная интеграция с Pydantic v2 гарантирует типизированную валидацию входных данных — критично для API с 59 endpoint-ами [11]. Автогенерация интерактивной документации API (Swagger UI) по аннотациям типов упрощает тестирование без дополнительных зависимостей. Django, обладая зрелой экосистемой, избыточен: встроенная ORM (Object Relational Mapping), шаблонизатор и административная панель не используются, а шаблон MTV (Model-Template-View) привносит ограничения. Flask потребовал

бы ручной настройки асинхронности, валидации и документации через сторонние расширения.

Система управления базами данных. Выбор СУБД определялся историей проекта. На начальном этапе использовалась YDB — распределённая СУБД Яндекса в Yandex Cloud. Миграция на PostgreSQL 16 выполнена в феврале 2026 года по трём причинам:

- локальное развёртывание без зависимости от облачного провайдера;
- асинхронный драйвер `asyncpg` с высокой пропускной способностью;
- зрелость экосистемы ORM (SQLAlchemy 2.0 с полной поддержкой `async/await`).

Сравнение СУБД приведено в таблице 3.

Таблица 3

Сравнение СУБД для аналитической системы

Критерий	PostgreSQL 16	MySQL 8	YDB
Развёртывание	Docker (локально)	Docker (локально)	Yandex Cloud (SaaS, программное обеспечение как услуга)
Асинхронный драйвер	<code>asyncpg</code> (500K+ ins/s)	<code>aiomysql</code> (менее зрелый)	<code>ydb[yc]</code> (синхронный)
ORM	SQLAlchemy 2.0 (async)	SQLAlchemy 2.0 (async)	Нет (YQL-запросы)
JSON-поддержка	<code>jsonb</code> (индексируемый)	JSON (нет GIN-индексов (Generalized Inverted Index))	JSON (ограничено)
Оконные функции	Полный набор	Полный набор	Ограничены
Стоимость	Бесплатно	Бесплатно	Платно (после бесплатного тарифа)

PostgreSQL 16 работает с `asyncpg` — драйвером, демонстрирующим пропускную способность свыше 500 тысяч вставок в секунду в эталонных тестах производительности (бенчмарках) MagicStack [12]. Индексируемый `jsonb` хранит метаданные отелей переменной структуры. YDB при всех достоинствах распределённой архитектуры оказалась избыточной для масштаба проекта

(менее 40 тысяч записей) и создавала единую точку отказа в виде облачного сервиса.

SQLAlchemy 2.0 используется как ORM-слой с асинхронными сессиями. Модель данных включает пять таблиц: Hotel, HotelStatistic, Event, QueryHistory, Forecast. Связи обеспечены внешними ключами с каскадным удалением, ограничения CHECK контролируют допустимые диапазоны значений (occupancy 0–100%, rating 0–10).

Кэширование и rate limiting. Redis 7 применяется в двух ролях: кэширование ответов ресурсоёмких endpoint-ов и ограничение частоты запросов по алгоритму скользящего окна (sliding window). Прогнозные endpoint-ы кэшируются с TTL (Time to Live) 30 минут, аналитические — 15 минут. Rate limiting ограничивает обращения к AI-агенту десятью запросами в минуту на IP-адрес (Internet Protocol, межсетевой протокол). При недоступности Redis система переключается на резервный механизм (fallback) в оперативной памяти на базе словарей Python [13].

Инфраструктура развёрнута через Docker Compose: PostgreSQL 16, Redis 7 и контейнеры приложения. Обратный прокси-сервер nginx обслуживает статические файлы, проксирует API-запросы и обеспечивает заголовки безопасности [14]. Детальная конфигурация инфраструктуры приведена в приложении Б.

Планировщик задач. Для периодического сбора данных выбран APScheduler — встраиваемый в процесс планировщик, не требующий отдельного брокера сообщений. Альтернатива — Celery — предполагает запуск отдельного рабочего процесса и брокера (RabbitMQ или Redis), что избыточно для единственного сервера с четырьмя периодическими задачами. При росте нагрузки миграция на Celery возможна без изменения логики задач.

Безопасность API реализована на четырёх уровнях [15]:

- аутентификация через API-ключ для парсерных конечных точек;
- ограничение частоты запросов (rate limiting) через Redis с резервным механизмом в оперативной памяти;

- типизированная валидация входных данных через Pydantic;
- защита от SQL-инъекций: экранирование спецсимволов в LIKE выражениях.

Детальное описание механизмов безопасности вынесено в приложение Б.

Перечисленные серверные технологии образуют инфраструктуру обработки данных — конвейер от сбора и хранения до прогнозирования и кэширования результатов. Не менее значима клиентская часть, отвечающая за визуализацию результатов и взаимодействие с пользователем.

Клиентская часть. Выбор фреймворка для пользовательского интерфейса определялся тремя факторами: экосистема визуализации данных, поддержка TypeScript и доступность компонентов для дашбордов. Рассмотрены три варианта (таблица 4).

Таблица 4

Сравнение клиентских фреймворков (фронтенд-фреймворков)

Критерий	React 18	Vue 3	Svelte 5
Экосистема визуализации	Recharts, ECharts, Nivo, Victory	ECharts (vue-echarts)	Ограничена (LayerCake)
Типизация	TypeScript (нативная)	TypeScript (Composition API)	TypeScript (ограниченная)
Рынок (State of JS 2025)	82% usage rate	51% usage rate	21% usage rate
SSE/потокосая передача (стриминг)	Нативный fetch + EventSource	Аналогично	Аналогично
Управление состоянием	TanStack Query, Zustand	Pinia	Встроенные stores

React 18 выбран благодаря доминирующей экосистеме компонентов для визуализации данных [16]. Recharts и ECharts предоставляют полный набор типов диаграмм, необходимых для аналитического дашборда: AreaChart, BarChart, LineChart, RadarChart, Treemap, GeoMap. Vue 3 с vue-echarts покрывает часть потребностей, но отсутствует аналог Recharts с декларативным JSX синтаксисом (JavaScript XML). Svelte 5, при элегантности реактивной модели, не

располагает зрелой библиотекой для географических визуализаций, что критично для страницы «Регионы и карта».

Сборщик Vite 7 выбран вместо webpack по причине скорости: dev-сервер стартует за 300 мс благодаря нативной поддержке ES-модулей, горячая замена модулей (Hot Module Replacement, HMR) обновляет компонент за 50 мс [17]. Webpack в сопоставимой конфигурации требует 8–12 секунд на холодный старт и 1–2 секунды на HMR, что существенно замедляет итеративную разработку интерфейса. Итоговая сборка (production) Vite включает удаление неиспользуемого кода (tree-shaking), разделение на фрагменты по маршрутам (code splitting) и минификацию.

Tailwind CSS (Cascading Style Sheets, каскадные таблицы стилей) 4 — утилитарный CSS-фреймворк, генерирующий стили на основе используемых классов [18]. Утилитарный подход (utility-first) позволяет строить дизайн-систему на CSS-переменных, что гарантирует визуальную согласованность всех компонентов. Тёмная и светлая темы реализованы через переопределение переменных без дублирования стилей. Набор компонентов — Button (четыре варианта), Card, Badge, Input, Dropdown, ErrorState — образует атомарную дизайн-систему с единообразными радиусами скругления, тенями и отступами. Данная структура следует методологии Atomic Design, разработанной Б. Фростом [19]. Методология выстраивает иерархию компонентов по пяти уровням: атомы (минимальные элементы — кнопка, поле ввода), молекулы (функциональные группы атомов — например, строка поиска из кнопки и текстового поля), организмы (самостоятельные секции интерфейса — панель КРІ, блок фильтров), шаблоны (скелеты страниц без данных) и страницы (конкретные реализации шаблонов с реальным контентом). При добавлении новой аналитической страницы элементы не создаются с нуля — достаточно комбинации существующих атомов и молекул.

Визуализация данных. Для аналитической системы выбор библиотек графиков определяет качество восприятия данных пользователем — принцип

соотношения информативных элементов к общему объёму визуализации (data-ink ratio) Эдварда Тафти. Рассмотрены четыре варианта (таблица 5).

Таблица 5

Сравнение библиотек визуализации данных

Критерий	Recharts	ECharts	Chart.js	D3.js
Интеграция с React	Нативная (JSX)	Обёртка (echarts-for-react)	Обёртка (react-chartjs-2)	Низкоуровневая (ref)
Географические карты	Нет	GeoMap, точечная географическая диаграмма (scatter-geo)	Нет (плагин chartjs-geo)	Полная поддержка
Размер итогового пакета сборки (бандла)	~45 КБ (gzip)	~300 КБ (tree-shaking)	~65 КБ	~75 КБ (ядро)
Декларативность	Высокая	Конфигурационная (options)	Конфигурационная	Императивная
Типы диаграмм	Area, Bar, Line, Radar, Treemap	20+ типов, GeoJSON	Bar, Line, Pie, Radar	Неограниченно

Принято решение использовать две библиотеки совместно. Recharts применяется для 90% графиков (AreaChart, BarChart, LineChart, RadarChart, Treemap, ComposedChart) благодаря декларативному JSX-синтаксису и малому размеру [20]. ECharts подключён исключительно для GeoMap — пузырьковой карты отелей с GeoJSON-контуром Иркутской области на странице «Регионы и карта» [21]. Разделение обосновано: Recharts удобнее для типовых графиков, а ECharts — единственная из рассмотренных библиотек с полноценной поддержкой точечной географической диаграммы (scatter-geo) на произвольном формате географических данных GeoJSON без привлечения D3. Chart.js отклонён из-за отсутствия нативной поддержки географических карт и Treemap. D3.js потребовал бы ручной реализации каждого компонента — трудозатраты, не оправданные для прикладного проекта.

Методология проектирования архитектуры ИАС опирается на два взаимодополняющих принципа. Progressive disclosure (постепенное раскрытие) предполагает, что пользователь вначале получает упрощённое представление, а экспертные детали становятся доступны по запросу. Б. Шнейдерман формализовал этот подход в мантре визуальной аналитики: «сначала обзор, затем масштабирование и фильтрация, далее — детали по требованию» [22]. Восьмистраничная структура пользовательского интерфейса системы реализует оба принципа: «Главная» предоставляет обзорную сводку по району, «Аналитика» позволяет масштабировать и фильтровать данные через KPI и временные ряды, «Карточка отеля» раскрывает детали конкретного объекта размещения. Такая последовательность снижает когнитивную нагрузку — пользователь не сталкивается с полным объёмом данных на первом экране, а погружается в аналитику постепенно.

Управление серверным состоянием реализовано через TanStack React Query [23]. Библиотека берёт на себя кэширование ответов API, ревалидацию при возврате фокуса окна, повторные попытки при сетевых ошибках и дедупликацию параллельных запросов. Время жизни кэша дифференцировано: дольше для справочных данных (отели, события), короче для аналитических (прогнозы, KPI) и без кэширования для чат-запросов. Это устраняет типовые проблемы рассогласования данных между компонентами.

1.3 Обоснование выбора методов прогнозирования и фреймворков реализации интеллектуального агента

Задача прогнозирования заполняемости средств размещения формулируется как предсказание процента занятых номеров на горизонте 3–90 дней с учётом календарных, погодных и событийных факторов. Ряд заполняемости характеризуется выраженной недельной сезонностью (пик в пятницу–субботу), годовой сезонностью (зима и лето), влиянием праздников и событий, а также потенциальными пропусками в наблюдениях. Объём накопленных данных — около года; детализация по числу записей приведена при описании базы данных. Это ограничивает применимость ряда методов.

Классические статистические методы. ARIMA и её сезонное расширение SARIMA моделируют ряд через авторегрессию, дифференцирование и скользящее среднее. Подбор порядков (p , d , q) и сезонных параметров (P , D , Q)s требует анализа автокорреляционной функции, что трудоёмко при масштабировании на 15 районов. Базовая формулировка не включает внешние регрессоры; расширение SARIMAX (SARIMA with exogenous variables, сезонная модель с экзогенными переменными) допускает их, но при коротких рядах возрастает риск переобучения. Метод Хольта-Уинтерса поддерживает только одну сезонную частоту. На практике эти методы протестированы, но отвергнуты из-за невозможности штатной интеграции погоды, событий и праздников без построения отдельных конвейеров для каждого района.

Prophet. Модель Meta (Facebook) представляет ряд как аддитивную декомпозицию (1), где $g(t)$ — тренд, $s(t)$ — сезонные компоненты (ряды Фурье), $h(t)$ — эффект праздников, $\varepsilon(t)$ — шум [24]. Встроенная поддержка российских праздников (через библиотеку holidays) и внешних регрессоров (temperature, precipitation из Open-Meteo) делает модель пригодной для данного проекта без дополнительных инженерных усилий. Prophet устойчив к пропускам, не требует стационарности и автоматически определяет точки перелома тренда.

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t) \quad (1)$$

NeuralProphet. Расширение Prophet на базе PyTorch, добавляющее нейросетевой компонент авторегрессии (AR Network) и лаговые регрессоры [25]. В отличие от Prophet, NeuralProphet принимает лаговые значения температуры через add_lagged_regressor, обучая индивидуальные веса для каждого лага. Модель улавливает задержку влияния погоды на спрос: прогноз хорошей погоды за 3–5 дней увеличивает число бронирований.

XGBoost и LightGBM. Модель XGBoost основана на методе градиентного бустинга решающих деревьев, реализованном в одноимённом модуле Python [26]. Прогноз формируется как сумма слабых моделей (2). В системе модуль FeatureEngineeringService генерирует 38 признаков восьми категорий: календарные, праздничные, лаговые, скользящие статистики, погодные,

событийные, трендовые и ценовые. Помимо стандартной модели, XGBoost обучается с квантильной регрессией для построения доверительных интервалов без параметрических предположений. LightGBM — альтернативная реализация градиентного бустинга со стратегией построения по листьям (leaf-wise), вносящей разнообразие в прогнозы.

$$F(x) = \sum f_k(x) \tag{2}$$

Ансамблевый подход. Отдельные модели обладают различными сильными сторонами: Prophet точнее на гладких рядах с выраженной сезонностью, NeuralProphet — на рядах с автокорреляцией благодаря лаговым регрессорам, XGBoost — при информативных событийных и погодных признаках. Ансамбль объединяет прогнозы методом inverse-RMSE (3), что присваивает больший вес более точной модели. Теоретическое обоснование ансамблирования состоит в снижении дисперсии: объединение некоррелированных прогнозов уменьшает общую ошибку даже при отсутствии улучшения смещения. Экспериментальная оценка качества моделей на данных Иркутской области приведена в разделе 2.2.

$$w_i = \frac{\frac{1}{RMSE_i}}{\sum_j \left(\frac{1}{RMSE_j} \right)} \tag{3}$$

Фреймворк интеллектуального агента. Для диалогового интерфейса рассмотрены три фреймворка (таблица 6).

Таблица 6

Сравнение фреймворков AI-агентов

Критерий	LangGraph	LangChain AgentExecutor	CrewAI
1	2	3	4
Архитектура	Граф состояний (StateGraph)	Цепочка вызовов	Мультиагентные роли
Контроль потока	Полный (условные рёбра, Command)	Ограниченный	Ограниченный (role-based)
Память	Встроенное сохранение контекста (MemorySaver)	Внешняя (ConversationBuffer)	Внешняя

1	2	3	4
Потоковая передача	Нативная (stream_events)	Обратные вызовы (callbacks)	Ограничена
Применимость	Один агент с ветвлением	Простые цепочки	Мультиагентные сценарии

LangGraph выбран по двум причинам [27]. Архитектура StateGraph реализует нетривиальную маршрутизацию: агент анализирует запрос, выбирает инструменты, выполняет их и решает — сформировать ответ или запросить дополнительные данные. Встроенный механизм сохранения контекста (MemorySaver) хранит историю диалога по идентификатору сессии (thread_id). CrewAI ориентирован на мультиагентные сценарии с ролевым разделением, что избыточно для единственного аналитического агента.

В системе два агента: основной (main_agent) с пятью инструментами (search_hotels, search_events, get_weather, forecast_occupancy, get_statistics) и агент прогнозов (forecast_agent), формирующий объяснения через Pydantic-структурированный выход LLM.

Векторная база данных. Для RAG-конвейера (Retrieval-Augmented Generation) выбран ChromaDB — локальная векторная база данных с персистентным хранением на SQLite и фильтрацией по метаданным. Фильтрация критична для запросов вида «события в Иркутском районе за март». Альтернатива FAISS не обеспечивает персистентности и фильтрации по полям; Pinecone исключён из-за облачной зависимости. В системе проиндексировано свыше тысячи документов.

Выбор LLM-провайдера. Система поддерживает шесть провайдеров (Mistral, GigaChat, Groq, DeepSeek, OpenRouter, Gemini) с переключением через единственную переменную окружения LLM_PROVIDER. Mistral выбран основным провайдером: 1 миллиард токенов в месяц на бесплатном тарифе, качественная русскоязычная генерация модели mistral-large-latest (128K контекст), французская юрисдикция [28]. Groq и Gemini требуют VPN (виртуальную частную сеть) для доступа из РФ — при тестировании обнаружена

блокировка Groq с российских IP через Cloudflare [29]. GigaChat используется для эмбедингов русскоязычных текстов (GigaChatEmbeddings от Сбера) — модель оптимизирована для русского языка и показывает лучшее качество семантического поиска по описаниям отелей по сравнению с мультязычными моделями.

Выводы по главе

Исследование предметной области мониторинга туристической активности выявило три проблемы: фрагментацию данных по десяткам агрегаторов без открытых API, отсутствие интеграции событийных и метеорологических факторов в существующих платформах, недоступность коммерческих RMS для малого гостиничного бизнеса Прибайкалья. Анализ 30 российских агрегаторов подтвердил, что 101Hotels — единственный сервис с открытым JSON API. Восемь парсеров событий и Open-Meteo API покрывают недостающие источники данных.

Технологический стек сформирован на основе сравнительного анализа: FastAPI для асинхронной серверной части (бэкенда) с 59 endpoint-ами, PostgreSQL 16 с asyncpg для локального развёртывания, React 18 с TypeScript для клиентской части с аналитическими панелями. Recharts и ECharts обеспечивают визуализацию 90% графиков и географическую карту соответственно. Tailwind CSS 4 формирует атомарную дизайн-систему с поддержкой тёмной и светлой тем.

Классические методы прогнозирования (ARIMA, Holt-Winters) отвергнуты из-за отсутствия штатной интеграции экзогенных переменных. Обоснован ансамблевый подход, объединяющий Prophet, NeuralProphet и XGBoost с адаптивными весами inverse-RMSE для снижения дисперсии прогноза. Интеллектуальный агент на LangGraph с пятью инструментами и RAG-конвейером на ChromaDB с эмбедингами GigaChat предоставляет аналитику на естественном языке. Mistral Large выбран основным LLM-провайдером благодаря лимиту в 1 миллиард токенов и качественной русскоязычной генерации.

2 РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ ПРОГНОЗИРОВАНИЯ ЗАПОЛНЯЕМОСТИ СРЕДСТВ РАЗМЕЩЕНИЯ

2.1 Архитектура системы и подсистема сбора данных

Проектируемая информационно-аналитическая система (ИАС) мониторинга туристической активности Иркутской области построена по клиент-серверной архитектуре с модульным серверным ядром. Выбор монолитной модульной архитектуры, а не микросервисной, обусловлен масштабом проекта: система обслуживает один регион с прогнозируемой нагрузкой до нескольких тысяч запросов в сутки, и накладные расходы на оркестрацию контейнеров, сервисную шину и распределённые транзакции не оправданы. Модульность достигается разделением серверной части на изолированные слои: маршрутизаторы запросов (роутеры), бизнес-логика (services), доступ к данным (db), модели данных (models), парсеры (parsers), промежуточное ПО (middleware) и зависимости (dependencies). Три основных слоя архитектуры — клиентское SPA-приложение, модульное серверное ядро и инфраструктурные сервисы (PostgreSQL, Redis, ChromaDB) — показаны на рисунке 1.

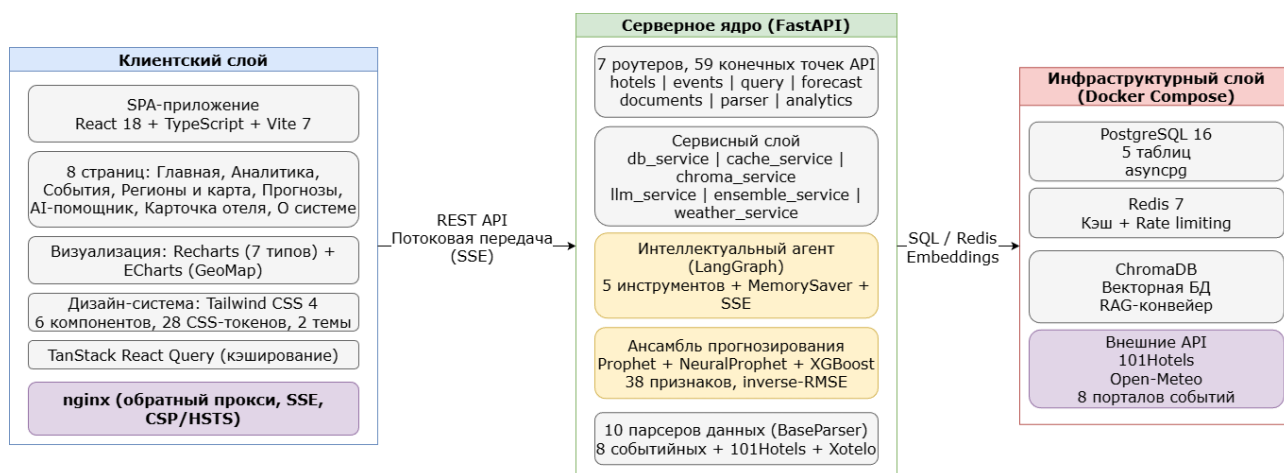


Рисунок 1 – Общая архитектура информационно-аналитической системы

Исторически на этапе курсовой работы использовалась Yandex YDB Serverless — облачная документо-ориентированная СУБД. В ходе производственной практики принято решение о миграции на PostgreSQL 16 для обеспечения возможности локальной разработки, внедрения полноценного ORM

и достижения независимости от облачного провайдера. Были переписаны ORM-модели на SQLAlchemy 2.0, заменён драйвер с YDB SDK на asyncpg, схема данных приведена к реляционной модели с внешними ключами и CHECK-ограничениями. Время отклика на типовых запросах сократилось примерно втрое по сравнению с облачной YDB, а локальное развёртывание через Docker Compose устранило зависимость от внешнего облачного провайдера.

Серверная часть реализована на FastAPI — асинхронном Python-фреймворке, обоснование выбора которого приведено в разделе 1.2. Фреймворк нативно поддерживает async/await, что критично для системы с интенсивным вводом-выводом: одновременные HTTP-запросы к внешним API, обращения к базе данных и вызовы LLM-провайдеров. Серверная часть содержит 7 роутеров, объединяющих 59 конечных точек API:

- hotels (3) — список, детали и статистика отелей;
- events (6) — каталог и управление событиями;
- query (5) — AI-агент и потоковая передача;
- forecast (9) — ансамблевый и покомпонентный прогнозы;
- documents (5) — управление документами RAG-конвейера;
- parser (12) — запуск и мониторинг парсеров;
- analytics (17) — KPI, корреляции, тепловая карта, данные для визуализаций;
- 2 корневых (health, root).

Роутер analytics предоставляет endpoint для дашборда ключевых показателей эффективности (KPI, Key Performance Indicators), тепловой карты заполняемости, корреляционного анализа, агрегированных данных по отелям и районам (в том числе для картографической визуализации на странице «Регионы и карта») и сравнения районов.

Клиентская часть — одностраничное приложение (SPA) на React 18 с TypeScript, взаимодействующее с сервером через REST API и Server-Sent Events (SSE) для потоковых ответов интеллектуального агента [30]. Обратный прокси-сервер nginx выполняет маршрутизацию запросов, сжатие статических ресурсов

и поддержку SSE-соединений. Политика общего происхождения ресурсов (CORS, Cross-Origin Resource Sharing) ограничивает источники запросов доменом клиентской части (фронтенда).

Инфраструктурный слой включает три компонента:

- PostgreSQL 16 — основная СУБД для хранения отелей, статистики и событий;
- Redis 7 — кэширование ответов и ограничение частоты запросов;
- ChromaDB — хранение векторных представлений документов для RAG-конвейера.

Все инфраструктурные компоненты развёрнуты в Docker-контейнерах через Docker Compose [31]. Конфигурация предусматривает два режима: по умолчанию запускаются только PostgreSQL и Redis (для локальной разработки серверной и клиентской частей); профиль full добавляет контейнеры серверной и клиентской частей для развёртывания в рабочем окружении. Контейнеры оснащены проверками работоспособности (healthcheck), секреты передаются через переменные окружения, процессы запускаются от непривилегированного пользователя (non-root) для снижения поверхности атаки.

Схема базы данных спроектирована на основе реляционной модели и реализована средствами SQLAlchemy 2.0 ORM с асинхронным драйвером asynpg [32, 33]. Асинхронный драйвер обрабатывает несколько запросов к БД параллельно, не блокируя цикл обработки событий FastAPI. База содержит пять таблиц с чётко определёнными связями и ограничениями целостности. Структура таблиц и их взаимосвязи отражены на рисунке 2.

Таблица Hotel — справочник средств размещения. На март 2026 года содержит 1 366 записей по 15 районам области. Каждый отель характеризуется названием, городом, районом, географическими координатами, рейтингом (шкала 0–10), минимальной ценой и типом размещения (25 категорий: гостиница, хостел, база отдыха, апартаменты, гостевой дом, санаторий и др.). Перекрёстный парсер Hotelo дополняет записи рейтингами и количеством отзывов TripAdvisor.

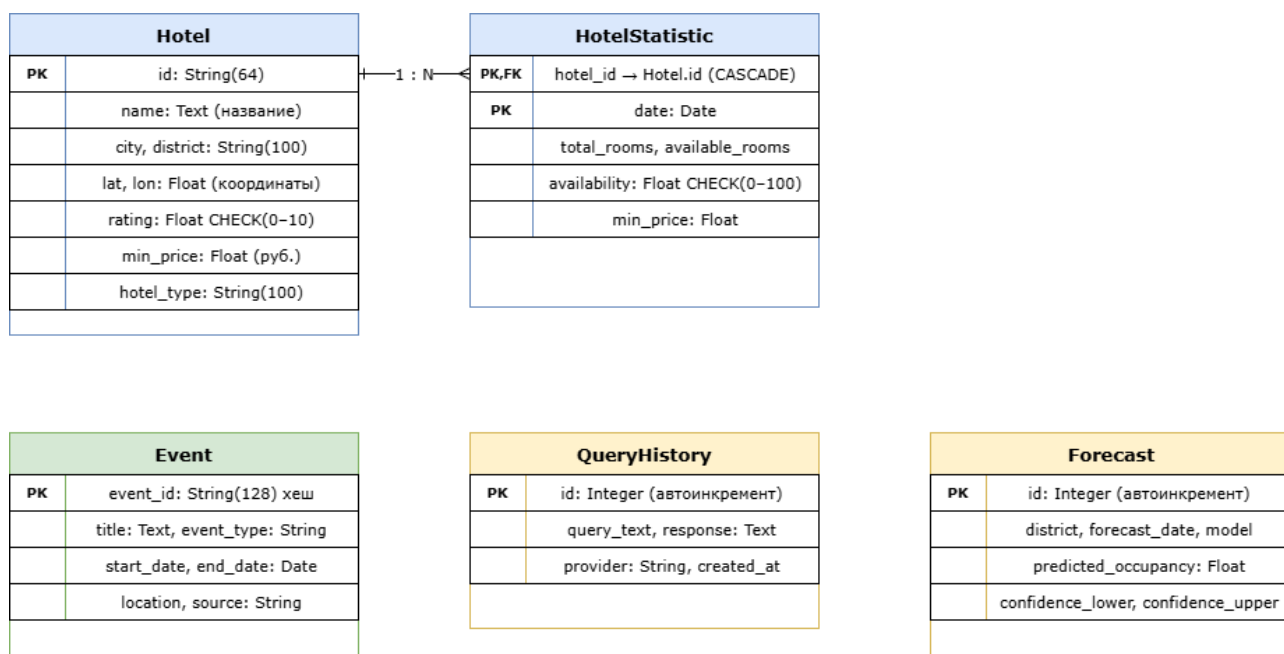


Рисунок 2 – Схема базы данных

Временные ряды заполняемости номерного фонда сосредоточены в HotelStatistic. Каждая запись привязана к конкретному отелю и дате (составной первичный ключ). При удалении отеля вся его статистика очищается автоматически (каскадное удаление). Фиксируются: общее количество номеров, число свободных, процент доступности, минимальная цена и вместимость. Дата последнего обновления отслеживается автоматически на уровне СУБД.

Культурные и туристические мероприятия региона агрегированы в Event. На март 2026 года — 376 записей. Первичный ключ формируется хешированием заголовка, даты начала и идентификатора источника, что гарантирует идемпотентность: одно и то же событие из разных порталов не создаст дубликат. Каждое событие описывается названием, датами проведения, типом («концерт», «выставка», «фестиваль», «экскурсия» и др.), местом и ссылкой на источник.

Обращения к интеллектуальному агенту фиксируются в QueryHistory: текст запроса, ответ агента и имя использованного LLM-провайдера. Временные метки формируются автоматически на уровне СУБД.

Результаты прогнозирования хранятся в Forecast: район, дата, предсказанная заполняемость, имя модели и границы доверительного интервала.

Хранение прогнозов позволяет ретроспективно сравнивать качество моделей и отдавать кэшированные результаты без повторных вычислений.

Подсистема сбора данных решает три задачи: извлечение информации о мероприятиях из восьми источников, получение статистики средств размещения с агрегатора 101Hotels и загрузка метеорологических данных из Open-Meteo. Все парсеры наследуют базовый класс BaseParser, определяющий единый интерфейс и общую логику [34].

BaseParser реализует единый интерфейс для всех парсеров: управление HTTP-сессиями, автоматические повторные попытки с экспоненциальной задержкой при ошибках сервера, ротацию заголовка User-Agent для противодействия блокировке (anti-detection). Конфигурация парсеров (таймауты, число попыток, задержки) вынесена в отдельный объект настроек.

Восемь парсеров событий покрывают основные информационные источники: irk.ru (городской портал Иркутска, HTML-парсинг через BeautifulSoup), culture38 (сайт министерства культуры Иркутской области), zeroevent (афиша мероприятий), yandex (Яндекс.Афиша), kassir (билетный сервис), culture_rf (федеральный портал «Культура.РФ»), telegram (каналы через Telethon API) и major (крупные площадки). Каждый возвращает унифицированный список событий с обязательной валидацией: название, дата начала и тип мероприятия проверяются при парсинге. Telegram-парсер представляет особый случай: сообщения из каналов содержат не только анонсы мероприятий, но и поздравления, рекламу, новости. Реализована LLM-классификация сообщений, отсеивающая нерелевантный контент (поздравления, реклама) на этапе парсинга.

Парсер средств размещения работает с JSON API агрегатора 101Hotels.com [35]. Конечная точка `ssg.101hotels.com/hotel/available/city/russia/{slug}` принимает символьный идентификатор города (slug) и возвращает JSON с информацией о доступности номеров для конкретного населённого пункта. 101Hotels выбран по итогам анализа агрегаторов, описанного в разделе 1.1. Парсер последовательно обходит

15 районов Иркутской области: Иркутский, Ольхонский, Слюдянский, Ангарский, Листвянка (Иркутский район), Байкальск (Слюдянский район) и другие. Для перекрёстной верификации данных реализован дополнительный парсер Hotelo, получающий рейтинги и количество отзывов TripAdvisor путём перекрёстной сверки (cross-reference) по названию и координатам отеля.

Метеорологические данные поступают из Open-Meteo API, описанного в разделе 1.1. Сервис предоставляет бесплатный доступ без ключа авторизации с лимитом 10 000 запросов в день. Запрашиваются суточные параметры: максимальная и минимальная температура воздуха, осадки, скорость ветра и код погоды по классификации WMO (World Meteorological Organization). Координаты запросов соответствуют центрам 15 районов. Горизонт прогноза погоды — 14 дней.

Автоматизация сбора реализована через APScheduler с асинхронным планировщиком AsyncIOScheduler [36]. Четыре периодические задачи запускаются по расписанию:

- средства размещения — каждые 2 часа;
- события из восьми источников — каждые 6 часов;
- метеорологические данные — каждые 3 часа;
- Telegram-каналы — каждый час.

Периодичность определена экспериментально: интервал в 2 часа для отелей достаточен для фиксации внутридневных колебаний доступности без превышения лимитов API; 6-часовой интервал для событий достаточен, поскольку культурные мероприятия обновляются не чаще раза в сутки. При запуске планировщик выполняет начальный сбор всех типов данных, затем работает по расписанию.

Нормализация данных выполняется на этапе парсинга. Названия районов приводятся к каноническому виду (например, «Иркутский район» в «Иркутский»). Дедупликация событий осуществляется по первичному ключу event_id, формируемому из хеша заголовка, даты и источника, — повторная вставка приводит к обновлению существующей записи, то есть выполняется

операция «вставка или обновление» (upsert). Для отелей применяется аналогичная стратегия через ON CONFLICT DO UPDATE в PostgreSQL. Пакетная вставка (batch insert) через ORM загружает до нескольких тысяч записей за одну транзакцию с автоматическим разрешением конфликтов (upsert).

Мониторинг парсеров отслеживает время последнего успешного запуска, количество собранных записей и процент ошибок. При деградации источника (изменение структуры страницы, блокировка) оператор получает уведомление через API статистики.

Доступ к данным централизован через сервисный слой, абстрагирующий операции чтения и записи для всех таблиц. Роутеры не содержат прямых SQL-запросов, что упрощает замену СУБД или добавление кэширующего слоя.

2.2 Модуль прогнозирования и интеллектуальный агент

Прогнозирование заполняемости средств размещения основано на ансамблевом подходе, объединяющем три модели: Prophet, NeuralProphet и XGBoost (LightGBM включён в XGBoost-сервис). Каждая модель обладает собственными достоинствами в отношении типов зависимостей, которые она способна моделировать. Объединение их прогнозов снижает дисперсию и повышает устойчивость результата. Конвейер ансамблевого прогнозирования показан на рисунке 3: исходные временные ряды проходят через модуль инженерии признаков (38 признаков), затем параллельно поступают в три модели, результаты которых объединяются весами inverse-RMSE в итоговый прогноз.

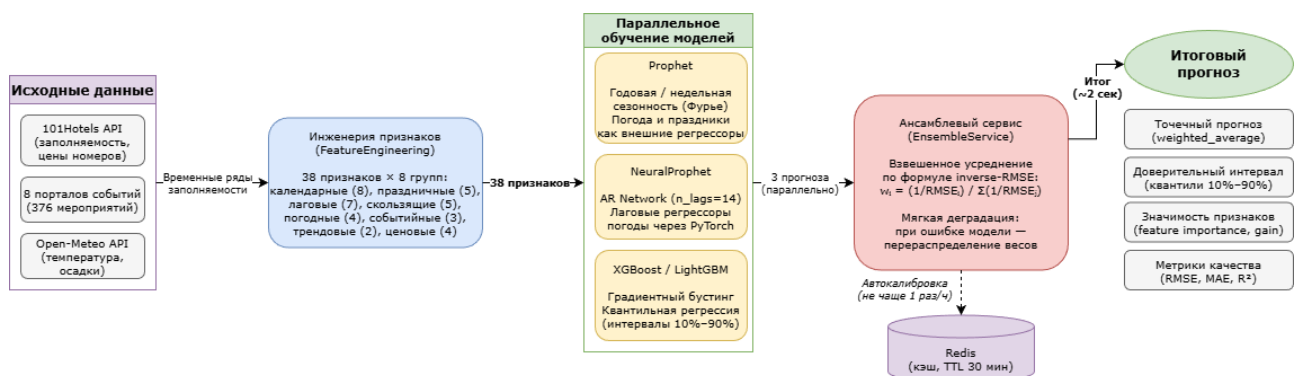


Рисунок 3 – Конвейер ансамблевого прогнозирования: от исходных данных через инженерию признаков к объединению прогнозов трёх моделей

Перед подачей данных в модели выполняется инженерия признаков (feature engineering) — процесс преобразования исходных временных рядов в набор числовых характеристик.

Модуль FeatureEngineering генерирует 38 признаков, разделённых на восемь групп: календарные (8), праздничные (5), лаговые (7), скользящие статистики (5), погодные (4), событийные (3), трендовые (2) и ценовые (4). Полный перечень признаков с указанием типа данных, диапазона значений и пояснения приведён в приложении Г.

Календарные признаки фиксируют день недели, месяц, квартал, номер недели в году, принадлежность к выходным и границам месяца. Заполняемость в пятницу и субботу в среднем на 12–18 п.п. выше, чем в середине рабочей недели. Праздничные признаки включают факт праздника, расстояние до ближайшего и от последнего праздника, принадлежность к длинным выходным и сезон. Под «длинными выходными» (is_long_weekend) понимается дата, являющаяся субботой или воскресеньем, либо приходящаяся на понедельник или пятницу при расстоянии до ближайшего государственного праздника не более одного дня — то есть рабочий день, «склеенный» с праздничным периодом и фактически образующий непрерывный отдых от трёх дней. Лаговые признаки моделируют автокорреляцию ряда заполняемости на горизонтах 1, 7, 14, 30 и 90 дней, а также разности первого порядка за 1 и 7 дней. Скользящие статистики — средние, стандартное отклонение, минимум и максимум в окнах 7 и 30 дней. Погодные признаки — температура, осадки, отклонение температуры от среднемесячной нормы и бинарный признак «хорошей погоды» (температура выше порога и низкие осадки). Корреляция между температурой и заполняемостью в Ольхонском районе летом достигает 0,42. Событийные признаки — количество мероприятий за день и неделю, наличие крупного события (концерт, фестиваль, спортивное мероприятие). Трендовые — порядковый индекс и нормализованный тренд. Ценовые — текущая стоимость номера, цена 7 дней назад, процентное изменение за неделю и скользящая средняя.

При отсутствии данных за конкретную дату применяется подстановка среднемесячных значений, рассчитанных по имеющимся историческим наблюдениям.

Реализация Prophet (модель описана в разделе 1.3) включает годовую и недельную сезонность с рядами Фурье. Российские праздники задаются явным списком с окном влияния в несколько дней до и после даты: система учитывает период, когда спрос начинает расти заранее. Температура и осадки подключены как внешние регрессоры через метод `add_regressor`. Обучение занимает 2–5 секунд на данных одного района.

В реализации NeuralProphet (раздел 1.3) параметр `n_lags` установлен в 14 — двухнедельное окно авторегрессии, подобранное экспериментально перебором значений от одной до четырёх недель. Погода и события подключены как лаговые регрессоры через `add_lagged_regressor`, а не как обычные: нейросеть обучает индивидуальные веса для каждого лага. Активирована ранняя остановка (`early_stopping`) для предотвращения переобучения. Время обучения — 10–15 секунд.

Модель XGBoost (описание в разделе 1.3) работает с 38 признаками модуля FeatureEngineering. Для построения доверительных интервалов обучаются три модели: стандартная (`squared error`) для точечного прогноза и две квантильные (`reg:quantileerror`, квантили 0,1 и 0,9) для нижней и верхней границ интервала 80%. Модель LightGBM обучается параллельно внутри того же сервиса.

Реализована квантильная регрессия (`quantile regression`) для построения доверительных интервалов: помимо стандартной модели XGBoost (`squared error`), обучается квантильная модель с функцией потерь `reg:quantileerror` для квантилей 0.1 и 0.9. Стандартная модель даёт точечный прогноз, квантили 0.1 и 0.9 — нижнюю и верхнюю границы 80-процентного доверительного интервала. Модель LightGBM реализована внутри XGBoost-сервиса как альтернативная реализация градиентного бустинга: стратегия построения по листьям (`leaf-wise` — выбор листа с максимальным снижением ошибки) отличается от

построения по уровням (level-wise) у XGBoost (равномерное углубление дерева), что вносит разнообразие в прогнозы и повышает устойчивость ансамбля.

Для оценки вклада отдельных признаков в прогноз реализована визуализация значимости признаков (feature importance) на странице «Прогнозы» в режиме «Модели». Горизонтальная столбчатая диаграмма (BarChart) отображает топ-10 признаков по метрике gain (суммарное снижение ошибки при использовании признака для разбиения) из моделей XGBoost и LightGBM. Визуализация позволяет аналитику оценить, какие факторы — календарные, погодные или событийные — оказывают наибольшее влияние на прогноз для конкретного района (рисунок 4).

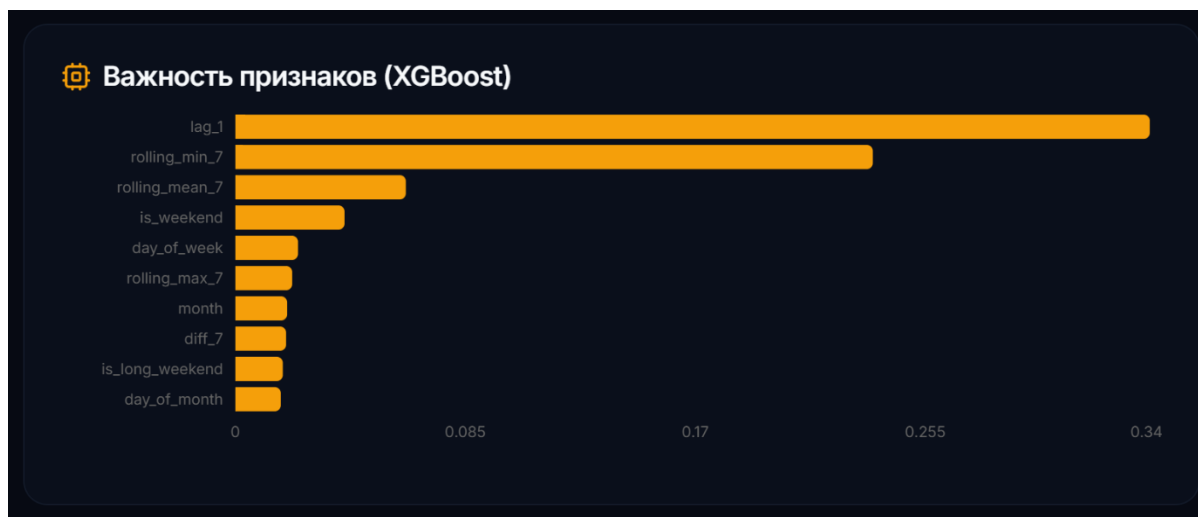


Рисунок 4 – Визуализация значимости признаков (feature importance) XGBoost в режиме «Модели» страницы «Прогнозы»

Ансамблевый сервис (EnsembleService) координирует запуск моделей и объединение результатов. Параллельное выполнение сокращает общее время прогноза до ~2 секунд вместо 15–20 при последовательном запуске: три модели запускаются одновременно в отдельных потоках. При ошибке одной модели остальные продолжают работу по принципу мягкой деградации (graceful degradation): если Prophet вернул пустой результат из-за нехватки данных, ансамбль формируется из оставшихся моделей с перераспределением весов.

Весы моделей в ансамбле рассчитываются по формуле inverse-RMSE, приведённой в разделе 1.3. Автокалибровка выполняется не чаще раза в час: сервис сравнивает модели на валидационных данных по метрикам RMSE, MAE

и R^2 , затем пересчитывает веса. Помимо взвешенного среднего (weighted_average) доступны простое среднее (simple_average) и выбор лучшей модели (best_model). Кэширование в Redis со временем жизни 30 минут исключает повторные вычисления при частых запросах.

Экспериментальная оценка качества моделей выполнена на данных трёх районов с наибольшим объёмом наблюдений. Горизонт прогноза — 7 дней, метрика — RMSE (Root Mean Square Error, среднеквадратичная ошибка) в процентных пунктах заполняемости. Результаты приведены в таблице 7.

Таблица 7

Метрики моделей прогнозирования (горизонт 7 дней)

Район	Prophet RMSE	NeuralProphet RMSE	XGBoost RMSE	Ensemble RMSE
Иркутский	13,72	6,43	12,19	3,85
Слюдянский	8,86	14,32	8,11	8,82
Ольхонский	12,11	13,25	19,68	6,08

Для Иркутского района ансамбль снижает RMSE до 3,85 при лучшем одиночном результате 6,43 (NeuralProphet) — улучшение на 40%. Для Ольхонского района ансамбль (6,08) значительно точнее каждой модели по отдельности (лучшая одиночная — Prophet, 12,11), что подтверждает эффективность комбинирования моделей различной природы. Слюдянский район демонстрирует случай, когда лучшая одиночная модель (XGBoost, 8,11) незначительно точнее ансамбля (8,82) — при перераспределении весов разница нивелируется. В целом ансамбль превосходит лучшую одиночную модель в двух из трёх районов, подтверждая выбранный подход.

Интеллектуальный агент построен на фреймворке LangGraph, обоснование выбора которого приведено в разделе 1.3. Агент реализован как StateGraph, в котором состояние включает историю диалога, контекст из базы знаний и счётчик вызовов инструментов для защиты от заикливания. Граф содержит два узла — model и tools — связанных условными рёбрами: если LLM-ответ содержит вызовы инструментов (tool_calls), управление передаётся в узел tools;

если нет — ответ возвращается пользователю. Схема графа состояний агента приведена (Рисунок 5).

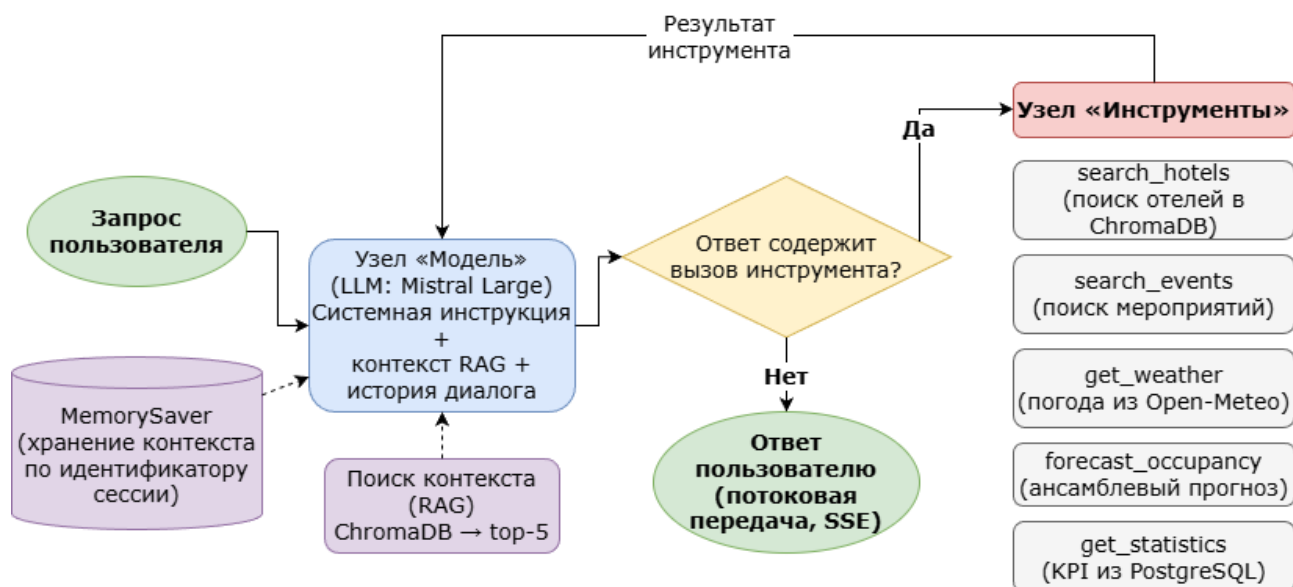


Рисунок 5 – Граф состояний интеллектуального агента (LangGraph StateGraph)

Пять инструментов покрывают основные аналитические потребности:

1) `search_hotels` — семантический поиск по ChromaDB, возвращает наиболее релевантные средства размещения с ценами, рейтингами и координатами;

2) `search_events` — поиск предстоящих мероприятий с фильтрацией по дате начала;

3) `get_weather` — обращение к Open-Meteo API за текущей и прогнозной погодой по координатам населённых пунктов области;

4) `forecast_occupancy` — делегирование отдельному агенту прогнозов, который собирает историю, запускает ансамбль и формирует объяснение на естественном языке;

5) `get_statistics` — агрегированные KPI из PostgreSQL: количество отелей, событий, средний рейтинг, заполняемость по районам.

RAG-конвейер (Retrieval-Augmented Generation) расширяет возможности агента контекстом из базы знаний [37]. Документы — описания отелей, события, статистические сводки — индексируются в ChromaDB с эмбедами GigaChat Embeddings (модель Embeddings API Сбера). При поступлении запроса текст

преобразуется в вектор, выполняется поиск ближайших соседей (top-5), найденные фрагменты конкатенируются и добавляются в системную инструкцию (промпт, prompt). Метаданные документов (тип, район, дата актуальности) позволяют сужать поиск до релевантного района и временного периода.

Основной LLM-провайдер — Mistral Large, доступный из РФ без виртуальной частной сети (VPN), с квотой 1 млрд токенов в месяц. Для выбора инструментов настроены минимальные параметры стохастичности, обеспечивающие стабильность ответов. Мультипровайдерная архитектура поддерживает переключение между провайдерами (GigaChat, Groq, DeepSeek, OpenRouter) через единственную переменную окружения; абстракция LangChain скрывает различия API провайдеров.

Потоковая передача через SSE реализована в endpoint POST /api/query/stream, возвращающий StreamingResponse с типом text/event-stream. Каждый токен LLM-ответа передаётся клиенту отдельным SSE-событием. Пользователь видит начало ответа менее чем через секунду, не дожидаясь полной генерации. Контекст разговора сохраняется в памяти агента: при следующем сообщении агент «помнит» предыдущие реплики, что позволяет вести связный диалог. При ошибке LLM-провайдера SSE-канал отправляет событие error и закрывает соединение.

2.3 Проектирование пользовательского интерфейса и обеспечение качества

Клиентская часть системы реализована на связке React 18 + TypeScript + Vite 7, обоснованной в разделе 1.2. TypeScript работает в строгом режиме, что исключает категорию ошибок, связанных с неявными типами. Маршрутизация выполнена через React Router 7 с ленивой загрузкой страниц, не требующихся при начальном отображении.

Структура навигации системы построена по задачному принципу: каждая из восьми страниц соответствует конкретному пользовательскому сценарию. Задачный подход к навигации означает, что каждая страница отвечает на один

ключевой вопрос: «Куда поехать?» («Главная»), «Что происходит в районе?» («Аналитика»), «Какие события запланированы?» («События»), «Как сравнить районы?» («Регионы и карта»). В ходе разработки навигация была реструктурирована: первоначальный вариант группировал страницы по типу данных (отели, события, погода), итоговый — по пользовательским ролям, разделяя туристические страницы с упрощённой подачей и аналитические с экспертным инструментарием. Подход основан на методологии Л. Розенфельда и П. Морвилля, определяющей информационную архитектуру интерфейса как проектирование систем организации, навигации и поиска контента, оптимизированных под ментальные модели целевой аудитории [38].

Назначение каждой страницы:

- 1) «Главная» — планирование поездки на Байкал (выбор района, периода, быстрые вопросы к AI-ассистенту);
- 2) «Аналитика» — аналитический обзор района с KPI, историей цен, рекомендациями и прогнозом на 14 дней;
- 3) «События» — каталог мероприятий с фильтрацией по типу, источнику и дате;
- 4) «Регионы и карта» — географическое сопоставление районов (карточки, тепловая карта, радарное сравнение);
- 5) «Прогнозы» — экспертное прогнозирование с переключением между простым и развёрнутым режимами;
- 6) «AI-помощник» — полноэкранный диалог с интеллектуальным агентом через потоковую передачу;
- 7) «Карточка отеля» — детальная страница отеля, доступная по прямой ссылке /hotels/:id;
- 8) «О системе» — презентация системы для защиты ВКР.

Основным элементом навигации пользовательского интерфейса является боковая панель, расположенная в левой части экрана. Панель содержит семь пунктов меню, каждый из которых включает графическую иконку, текстовое название раздела и краткое описание, поясняющее назначение страницы.

Пункты «Прогнозы» и «AI-помощник» дополнены визуальными метками «ML» и «AI» соответственно, сигнализирующими об использовании методов машинного обучения и интеллектуальной обработки данных, что повышает прозрачность системы для конечного пользователя. Помимо семи разделов меню, в системе присутствует восьмая страница — карточка отеля, которая не вынесена в навигацию, поскольку представляет собой детализированное описание конкретного объекта размещения. Переход на неё осуществляется через нажатие на отель из разделов «Аналитика» или «Регионы и карта», то есть от общего обзора к детальной информации. Структура навигации показана на рисунке 6.

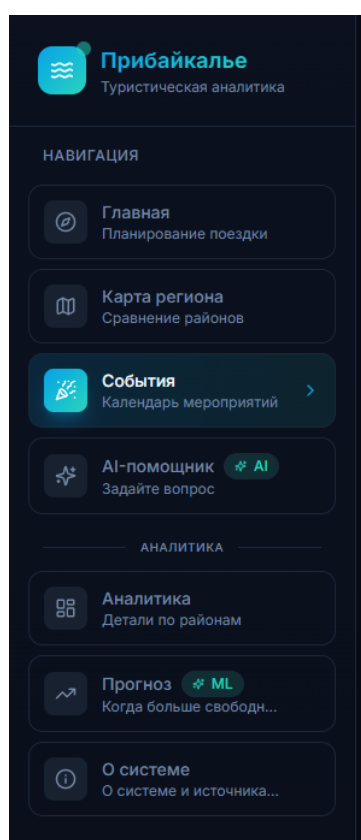


Рисунок 6 – Навигационная структура интерфейса: боковая панель с семью пунктами

Типичный сценарий взаимодействия туриста выстроен по принципу «от общего к частному». Пользователь попадает на «Главную», выбирает район и период поездки, получает сводку — заполняемость, погоду, предстоящие события. Интерес к конкретному району ведёт на «Аналитику», где KPI

карточки, история цен и корреляция «заполняемость–погода» дают панорамный обзор. При клике на отель из списка рекомендаций открывается «Карточка отеля» с графиками заполняемости и динамики цен. Если пользователь хочет задать уточняющий вопрос, ссылка из любой страницы ведёт в «AI-помощник» с передачей контекста текущего района. Для аналитика или менеджера предусмотрен альтернативный маршрут: «Регионы и карта» (сравнение районов), далее «Прогнозы» (экспертный режим с метриками моделей и значимостью признаков (feature importance)), далее «AI-помощник» (формулирование запроса на основе увиденных данных).

Принцип progressive disclosure реализован на странице «Прогнозы» наиболее явно. По умолчанию пользователь видит простой режим: один график ансамблевого прогноза с доверительным интервалом. Переключатель «Прогноз / Модели / Сезонность» открывает дополнительные слои: кривые отдельных моделей, таблицу метрик (RMSE, MAE, R^2), диаграмму значимости признаков XGBoost, AI-объяснение от ForecastAgent и возможность сравнить районы. Неподготовленный посетитель видит лишь итоговый график, аналитик — полный инструментарий.

Дизайн-система. Визуальная целостность интерфейса реализована через дизайн-систему на Tailwind CSS 4. Утилитарный подход (utility-first) задаёт стили классами непосредственно в JSX, исключая проблему специфичности CSS-селекторов. Компонентная библиотека содержит шесть базовых элементов, перечисленных в таблице 8.

Таблица 8

Компоненты дизайн-системы

Компонент	Варианты	Размеры	Назначение
1	2	3	4
Button	primary, secondary, ghost, outline, danger	sm, md, lg, icon	Действия пользователя, навигация
Card	default, glass, gradient, glow	padding: none, sm, md, lg	Контейнер данных, карточки KPI

1	2	3	4
Badge	default, primary, accent, success, warning, danger, outline, secondary	sm, md	Статусы, категории, метки ML/AI
Input / Textarea	—	—	Формы ввода, поиск
Dropdown	compact, default	с иконкой, без	Настраиваемый выпадающий список с клавиатурной навигацией
ErrorState	—	—	Отображение ошибок с кнопкой повтора

Цветовая палитра отражает тематику проекта. Основной акцент — оттенки голубого, заданные в формате HSL (Hue, Saturation, Lightness — тон, насыщенность, светлота): 199 89% 48%. Акцентный бирюзовый (172 66% 50%) ассоциируется с природой Прибайкалья. Функциональные цвета несут семантическую нагрузку: зелёный — положительная динамика, жёлтый — средняя заполняемость, красный — высокая заполняемость или ошибка. Разделение следует принципу досознательной обработки (pre-attentive processing): пользователь считывает статус визуально до осознанного чтения цифр. Цветовые решения опираются на эвристики юзабилити Дж. Нильсена и принципы визуального отображения данных Э. Тафти [39, 40].

Тёмная и светлая темы реализованы через 28 CSS-переменных, определяющих фон, цвет текста, акценты и градиенты. Тёмная тема включена по умолчанию; переключение сохраняет выбор в локальном хранилище браузера. Общий вид страницы «Аналитика» с дашбордом ключевых показателей района приведён на рисунке 6.

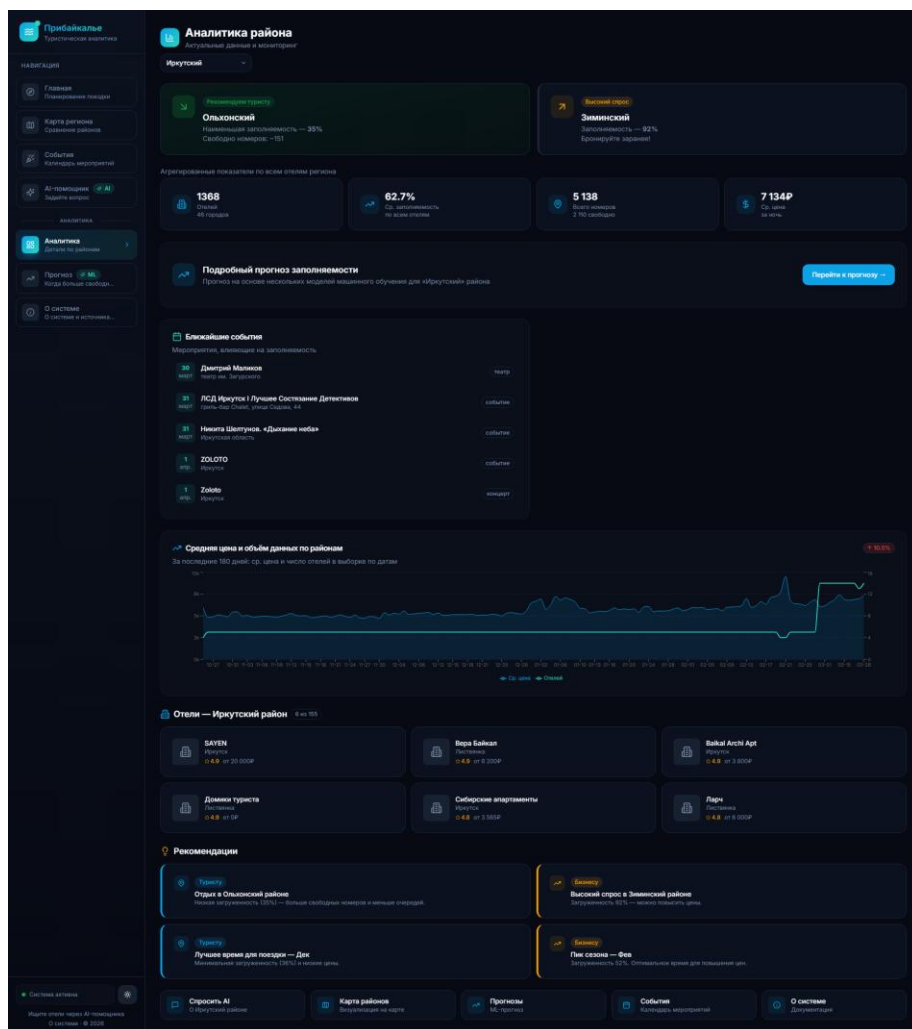


Рисунок 6 – Страница «Аналитика»: дашборд ключевых показателей района

Компонентный подход следует методологии атомарного дизайна Б. Фроста (раздел 1.2): атомы (кнопка, бейдж, поле ввода) комбинируются в молекулы (карточка отеля) и организмы (блок KPI, панель фильтров), из которых собираются страницы. Единообразные радиусы скругления, тени и отступы обеспечивают визуальную согласованность на экранах различного разрешения.

Визуализация данных. Выбор типа диаграммы для каждой аналитической задачи обусловлен характером данных и целевым восприятием пользователя. В системе применяются семь типов диаграмм:

- AreaChart — временной ряд заполняемости (процент занятых номеров) с областью доверительного интервала; применяется на страницах «Прогнозы», «Аналитика» и «Карточка отеля»;
- BarChart — категориальное сравнение: значимость признаков прогнозных моделей (метрика gain), распределение событий по источникам;

- LineChart — наложение кривых прогнозов отдельных моделей (Prophet, NeuralProphet, XGBoost) на одном временном горизонте;
- RadarChart — многокритериальное сравнение районов по трём метрикам: средняя заполняемость (%), общее число номеров, средняя стоимость номера за сутки (ADR, Average Daily Rate);
- Treemap — иерархическое распределение числа объектов размещения по населённым пунктам внутри района;
- HeatmapGrid — матрица «район × дата», в каждой ячейке которой цветом (пятиступенчатая шкала от зелёного к красному) закодирован процент занятых номеров; позволяет выявлять сезонные закономерности и аномальные всплески заполняемости;
- GeoMap (ECharts) — картографическая визуализация: точечные маркеры объектов размещения на контуре Иркутской области; размер маркера пропорционален числу свободных номеров, цвет кодирует уровень заполняемости по трёхступенчатой шкале.

Страница «Аналитика» объединяет ключевые метрики в едином дашборде; верхняя часть экрана содержит KPI-карточки, ниже — ComposedChart с двумя осями Y: средняя стоимость номера за сутки (ADR) как Area и число объектов размещения в выборке как Line, затем рекомендации через компонент InsightCard (варианты: positive, neutral, negative, info).

Цветовое кодирование заполняемости единообразно во всём интерфейсе. Тепловая карта использует пятиступенчатую шкалу от зелёного (низкая заполняемость) через бирюзовый, жёлтый и оранжевый к красному (свыше 80%). Географическая карта применяет упрощённую трёхступенчатую версию тех же цветов. Индикатор заполняемости на главной странице и в карточке отеля дополняет цвет текстовой меткой («низкая», «средняя», «высокая», «очень высокая»). Наглядная легенда расшифровывает шкалу. Единая шкала упрощает интерпретацию статуса на разных страницах интерфейса.

Главная страница («Планирование поездки») с выбором района и сводкой по району показана на рисунке 7.

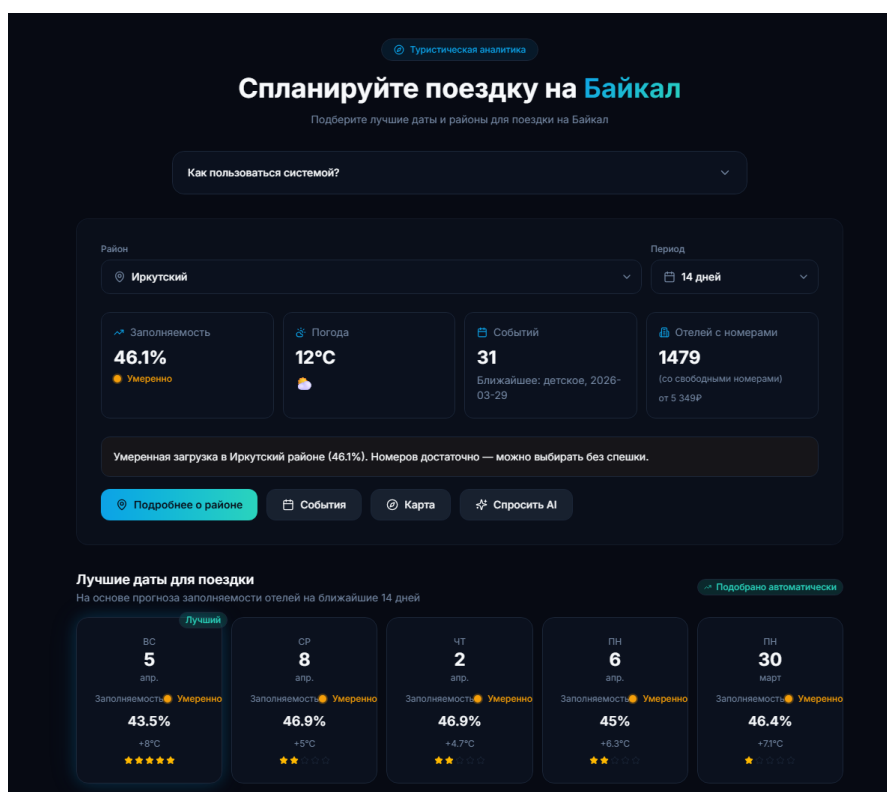


Рисунок 7 – Главная страница: планирование поездки и сводка по району

Страница «Прогнозы» визуализирует результаты ансамблевого прогнозирования (Рисунок 8). Основной график — AreaChart с двумя Area-слоями для верхней и нижней границ доверительного интервала и центральной линией ансамбля. Каждая модель окрашена в отдельный цвет: Prophet — голубой, NeuralProphet — фиолетовый, XGBoost — зелёный. Линия ансамбля выделена контрастным цветом переднего плана. Реализован экспорт графика в формат PNG для включения в отчёты и презентации.



Рисунок 8 – Страница «Прогнозы»: ансамблевый прогноз с доверительным интервалом и сравнение моделей

Раздел «Регионы и карта» предоставляет четыре режима просмотра: интерактивная карта (GeoMap), обзор карточек районов, тепловая карта (HeatmapGrid) и сравнение (radar, treemap, bar). GeoMap отображает точечные маркеры (scatter) отелей на упрощённом GeoJSON-контуре Иркутской области; размер точки пропорционален заполняемости, цвет кодирует уровень по трёхступенчатой шкале. Карта поддерживает масштабирование и подписи районов; подсказки содержат название, заполняемость, число номеров и цену. Для минимизации размера итогового пакета ECharts загружается с удалением неиспользуемого кода (tree-shaking): импортируются только необходимые модули. Общий вид страницы приведён на рисунке 9.

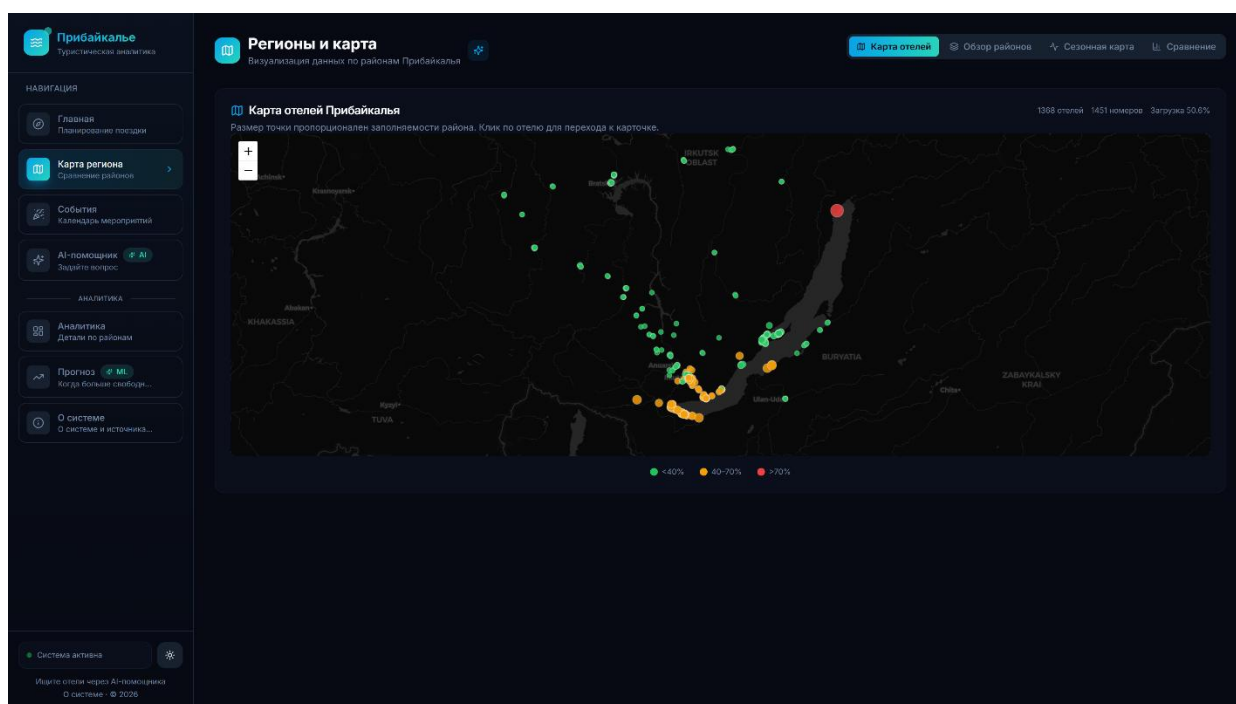


Рисунок 9 – Страница «Регионы и карта»: интерактивная GeoMap с отелями Иркутской области

Стиль всплывающих подсказок унифицирован на всех страницах: фон, граница и цвет текста привязаны к токенам текущей темы. Содержимое подсказки контекстуально: на графиках заполняемости — процент и дата, на диаграмме значимости признаков — числовой вес, на radar-диаграмме — округлённый процент.

Решения в области UX. Обработка ошибок стандартизирована через компонент `ErrorState`. Он содержит иконку, заголовок, описание и кнопку «Повторить». Компонент используется на всех страницах с серверными данными. Глобальная обёртка `ErrorBoundary` перехватывает критические ошибки рендеринга и предлагает перезагрузить страницу. Ошибки мутаций отображаются через всплывающие уведомления, стилизованные в соответствии с текущей темой.

Состояния загрузки реализованы через каркасные экраны-заглушки (скелетные экраны), а не через анимированные индикаторы вращения (спиннеры). CSS-класс для скелетонов создаёт мерцающую анимацию (`shimmer`) на основе цветовых токенов темы. Скелетоны воспроизводят структуру целевого контента: на странице «Аналитика» — прямоугольники вместо KPI-карточек и графиков, на «Прогнозах» — полоса вместо диаграммы с областями, на «Событиях» — ряды карточек, на «Регионах и карте» — область карты. Скелетный подход снижает воспринимаемое время ожидания: пользователь видит каркас интерфейса вместо пустого экрана и спиннера.

Страница «AI-помощник» реализует полноэкранный диалоговый интерфейс (Рисунок 10). Поточковая передача (SSE-стриминг) доставляет токены по мере их генерации; пользователь видит текст, появляющийся в реальном времени. Промежуточные вызовы инструментов агента отображаются информационными метками — «Ищу отели...», «Строю прогноз...» — и снимаются по завершении. История диалога сохраняется в локальном хранилище браузера (`localStorage`) под уникальным ключом сессии; при закрытии и повторном открытии вкладки последний диалог восстанавливается. Быстрые подсказки в виде интерактивных кнопок (чипов) снижают порог входа для пользователя, не знакомого с системой: «Лучшие отели на Ольхоне», «Прогноз заполняемости на майские». Интерфейс реализован по модели *conversational UI* — взаимодействие через диалог, а не через формы.

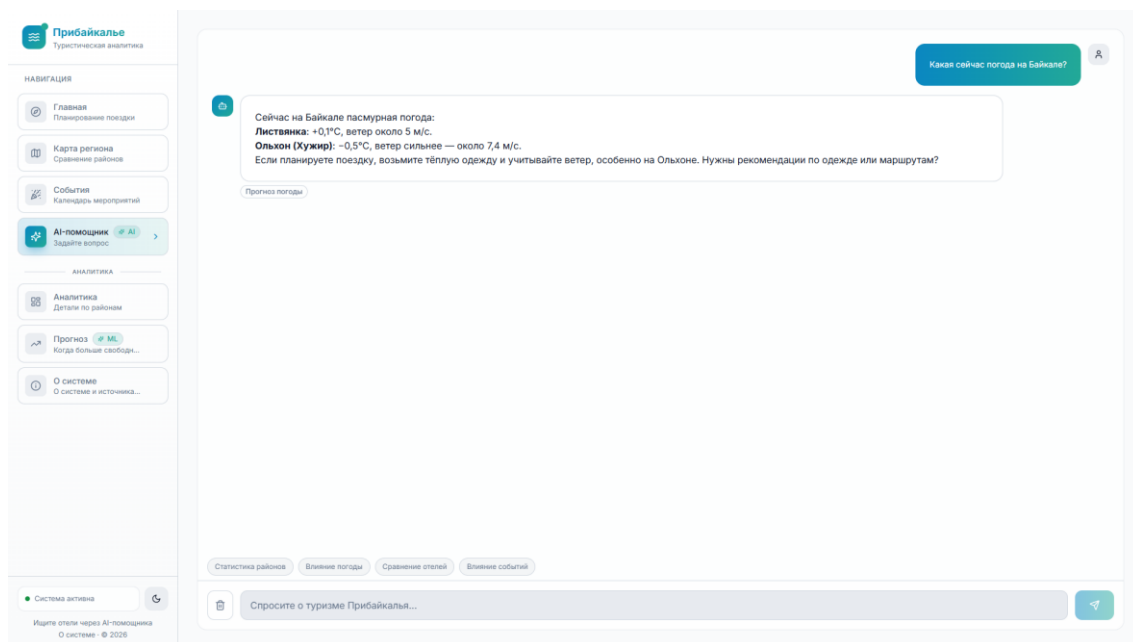


Рисунок 10 – Страница «AI-помощник»: диалог с интеллектуальным агентом

Адаптивная вёрстка реализована для трёх контрольных точек: мобильные устройства (до 640 px), планшеты (до 1024 px), десктоп (свыше 1024 px, расширенный сайдбар от 1536 px). На мобильных устройствах боковая панель скрывается за кнопкой меню в виде трёх горизонтальных линий (так называемая «гамбургер»-кнопка) в фиксированной верхней панели (хедере); при открытии полупрозрачное перекрытие (оверлей) блокирует основной контент. Кнопки и интерактивные элементы на мобильных имеют увеличенную область касания. Графики на Recharts занимают полную ширину контейнера через ResponsiveContainer; GeoMap на ECharts адаптирует высоту в зависимости от типа устройства.

Кэширование серверных данных на стороне клиента выполняется средствами TanStack React Query, описанной в разделе 1.2. В нижней части сайдбара индикатор отображает статус подключения к серверу — зелёная или красная точка.

Тестирование и безопасность. Тестирование охватывает два уровня. Модульные тесты (93 шт.) разделены на шесть файлов: ансамблевый сервис, вспомогательные функции прогнозирования, безопасность (rate limiting, аутентификация, валидация), роутеры API, схемы данных и инструменты агента. Тестовые заготовки (фикстуры) изолируют тесты: имитационные данные

(мок данные) истории заполняемости, тестовый Redis-клиент, пустая коллекция векторной базы. Тесты роутеров проверяют HTTP-контракты без запущенного сервера через встроенный асинхронный клиент. Сквозные (E2E) тесты включают 9 сценариев: health-endpoint, KPI, список отелей, события, прогноз Prophet, ансамблевый прогноз, погода, запрос к AI-агенту, статус планировщика. Тесты выполняются против запущенного сервера и реальной БД, проверяя цепочку от HTTP-запроса до JSON-ответа.

Четыре уровня безопасности API, описанные в разделе 1.2, реализованы в виде промежуточного ПО (middleware). Конкретные пороговые значения rate limiting, параметры валидации Pydantic и конфигурация заголовков CSP/HSTS вынесены в приложение Б.

Собранные данные содержат ограничение: летний период 2025 года (июль–сентябрь) не покрыт наблюдениями, поскольку система находилась на этапе проектирования. Пробел влияет на точность долгосрочных прогнозов — при горизонте 30+ дней R^2 может принимать отрицательные значения. Для демонстрации рекомендованы краткосрочные горизонты (3–14 дней). Архитектурно система полностью готова к работе с полными данными: при накоплении летних наблюдений в 2026 году модели обучатся на годовом цикле без изменений кода.

Выводы по главе

Выводы соотнесены с задачами работы. Проведён анализ предметной области и существующих решений (задача 1), обоснован выбор технологий и инструментов визуализации (задача 2), реализована подсистема автоматизированного сбора данных (задача 3), разработан ансамблевый метод прогнозирования заполняемости средств размещения и модуль его реализующий с визуальной аналитикой результатов (задача 4), спроектирована архитектура ИАС и реализована сама система, ее взаимодействие с интеллектуальным агентом и веб-интерфейс (задача 5).

ЗАКЛЮЧЕНИЕ

В ходе производственной практики спроектирована архитектура интеллектуальной аналитической системы (ИАС) и реализована сама система для мониторинга и прогнозирования туристической активности Иркутской области. Система решает практическую проблему отсутствия единого инструмента для оперативного анализа заполняемости средств размещения с учётом событийных, погодных и сезонных факторов.

Проведённое исследование предметной области выявило фрагментацию данных по десяткам агрегаторов без открытых программных интерфейсов. Анализ более 30 российских платформ подтвердил, что 101Hotels — единственный сервис с открытым JSON API. На основании сравнительного анализа сформирован технологический стек: FastAPI для серверной части, PostgreSQL 16 для хранения данных, React 18 с TypeScript для клиентской части, Recharts и ECharts для визуализации. Проектирование пользовательского интерфейса опиралось на принципы информационного дизайна Э. Тафти и эвристики юзабилити Дж. Нильсена.

Реализована подсистема автоматизированного сбора данных из 11 внешних источников: 10 парсеров с единым базовым классом и планировщиком APScheduler обеспечивают периодический сбор информации о средствах размещения, событиях и погоде. На март 2026 года база содержит 1 366 объектов размещения и 376 событий по 15 районам области.

Разработан ансамблевый метод прогнозирования заполняемости средств размещения и реализующий его модуль с визуальной аналитикой результатов. Метод объединяет Prophet, NeuralProphet и XGBoost с адаптивными весами inverse-RMSE. Среднеквадратичная ошибка ансамбля для Иркутского района составила 3,85 п.п. при лучшем одиночном результате 6,43 п.п., что подтверждает эффективность комбинирования моделей различной природы.

Успешно реализовано взаимодействие компонентов системы с интеллектуальным агентом на LangGraph и веб-интерфейс. Интерфейс из восьми страниц с дизайн-системой и семью типами диаграмм предоставляет

пользователям аналитику как в визуальном, так и в диалоговом формате. Качество системы подтверждено 93 модульными и 9 сквозными тестами, что свидетельствует о готовности решения к внедрению.

Основное ограничение — отсутствие данных за летний период 2025 года, снижающее точность долгосрочных прогнозов. Архитектурно ИАС готова к работе с полным годовым циклом без изменения кода. Перспективы развития включают расширение ансамбля за счёт LSTM и Transformer-моделей, географическое масштабирование на Бурятию и Забайкальский край, юзабилити-тестирование с представителями туристического бизнеса и развёртывание в облачной инфраструктуре с автомасштабированием.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Федеральная служба государственной статистики (Росстат): туризм — статистика по Иркутской области [Электронный ресурс], URL: <https://rosstat.gov.ru/> (дата обращения: 15.03.2026).
2. Стратегия развития туризма Иркутской области на период до 2030 года: утверждена распоряжением Правительства Иркутской области [Электронный ресурс], URL: <https://irkobl.ru/sites/tour/> (дата обращения: 15.03.2026).
3. Исполатов В.П. Анализ программных интерфейсов российских агрегаторов бронирования средств размещения: материалы исследования в рамках проекта. — Иркутск, 2025.
4. Паспорт туристского кластера Иркутской области: утверждён Агентством по туризму Иркутской области [Электронный ресурс], URL: <https://irkobl.ru/sites/tour/utp.php> (дата обращения: 25.03.2026).
5. STR Global: аналитика индустрии гостеприимства [Электронный ресурс], URL: <https://str.com/> (дата обращения: 18.03.2026).
6. AirDNA: аналитическая платформа данных краткосрочной аренды [Электронный ресурс], URL: <https://www.airdna.co/> (дата обращения: 25.03.2026).
7. Электронная путёвка: Единая информационная система электронных путёвок (ЕИСЭП) [Электронный ресурс], URL: <https://ev.economy.gov.ru/> (дата обращения: 25.03.2026).
8. Yandex DataLens: документация сервиса визуализации данных [Электронный ресурс], URL: <https://cloud.yandex.ru/docs/datalens/> (дата обращения: 20.03.2026).
9. Xotelo: Hotel Price Monitor and API [Электронный ресурс], URL: <https://xotelo.com/> (дата обращения: 22.03.2026).
10. Open-Meteo: бесплатный API погодных данных [Электронный ресурс], URL: <https://open-meteo.com/> (дата обращения: 22.03.2026).
11. FastAPI: документация фреймворка [Электронный ресурс], URL: <https://fastapi.tiangolo.com/> (дата обращения: 20.03.2026).

12. MagicStack/asyncpg: асинхронный драйвер PostgreSQL для Python [Электронный ресурс], URL: <https://github.com/MagicStack/asyncpg> (дата обращения: 20.03.2026).
13. Redis: документация [Электронный ресурс], URL: <https://redis.io/docs/> (дата обращения: 22.03.2026).
14. nginx: документация веб-сервера [Электронный ресурс], URL: <https://nginx.org/ru/docs/> (дата обращения: 21.03.2026).
15. OWASP: API Security Top 10 [Электронный ресурс], URL: <https://owasp.org/API-Security/> (дата обращения: 23.03.2026).
16. State of JavaScript 2025: ежегодное исследование экосистемы JavaScript [Электронный ресурс], URL: <https://stateofjs.com/> (дата обращения: 20.03.2026).
17. Vite: документация сборщика [Электронный ресурс], URL: <https://vite.dev/> (дата обращения: 20.03.2026).
18. Tailwind CSS: документация CSS-фреймворка [Электронный ресурс], URL: <https://tailwindcss.com/docs> (дата обращения: 20.03.2026).
19. Frost B. Atomic Design: методология проектирования интерфейсных систем [Электронный ресурс], URL: <https://atomicdesign.bradfrost.com/> (дата обращения: 25.03.2026).
20. Recharts: библиотека графиков для React [Электронный ресурс], URL: <https://recharts.org/> (дата обращения: 21.03.2026).
21. Apache ECharts: библиотека визуализации данных [Электронный ресурс], URL: <https://echarts.apache.org/> (дата обращения: 21.03.2026).
22. Shneiderman B. The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations //Proceedings of the IEEE Symposium on Visual Languages. - 1996. - P. 336-343.
23. TanStack Query: документация библиотеки управления серверным состоянием для React [Электронный ресурс], URL: <https://tanstack.com/query/> (дата обращения: 22.03.2026).

24. Prophet: документация библиотеки прогнозирования временных рядов [Электронный ресурс], URL: <https://facebook.github.io/prophet/> (дата обращения: 25.03.2026).

25. NeuralProphet: документация библиотеки нейросетевого прогнозирования [Электронный ресурс], URL: <https://neuralprophet.com/> (дата обращения: 25.03.2026).

26. XGBoost: документация библиотеки градиентного бустинга [Электронный ресурс], URL: <https://xgboost.readthedocs.io/> (дата обращения: 25.03.2026).

27. LangGraph: документация фреймворка [Электронный ресурс], URL: <https://langchain-ai.github.io/langgraph/> (дата обращения: 22.03.2026).

28. Mistral AI: документация API [Электронный ресурс], URL: <https://docs.mistral.ai/> (дата обращения: 23.03.2026).

29. Groq: платформа быстрого инференса LLM [Электронный ресурс], URL: <https://groq.com/> (дата обращения: 22.03.2026).

30. React: документация библиотеки [Электронный ресурс], URL: <https://react.dev/> (дата обращения: 20.03.2026).

31. Docker Documentation [Электронный ресурс], URL: <https://docs.docker.com/> (дата обращения: 21.03.2026).

32. PostgreSQL 16 Documentation [Электронный ресурс], URL: <https://www.postgresql.org/docs/16/> (дата обращения: 20.03.2026).

33. SQLAlchemy 2.0: документация ORM [Электронный ресурс], URL: <https://docs.sqlalchemy.org/en/20/> (дата обращения: 19.03.2026).

34. Python aiohttp: документация библиотеки асинхронных HTTP-запросов [Электронный ресурс], URL: <https://docs.aiohttp.org/> (дата обращения: 18.03.2026).

35. 101Hotels.com: агрегатор средств размещения [Электронный ресурс], URL: <https://www.101hotels.com/> (дата обращения: 25.03.2026).

36. APScheduler: документация библиотеки планирования задач [Электронный ресурс], URL: <https://apscheduler.readthedocs.io/> (дата обращения: 19.03.2026).

37. ChromaDB: документация векторной базы данных [Электронный ресурс], URL: <https://docs.trychroma.com/> (дата обращения: 21.03.2026).

38. Rosenfeld L., Morville P. Information Architecture for the World Wide Web. - 3rd ed. - Sebastopol: O'Reilly Media, 2006. - 504 p.

39. Nielsen J. 10 Usability Heuristics for User Interface Design [Электронный ресурс], URL: <https://www.nngroup.com/articles/ten-usability-heuristics/> (дата обращения: 25.03.2026).

40. Tufte E.R. The Visual Display of Quantitative Information. - 2nd ed. - Cheshire: Graphics Press, 2001. - 197 p.

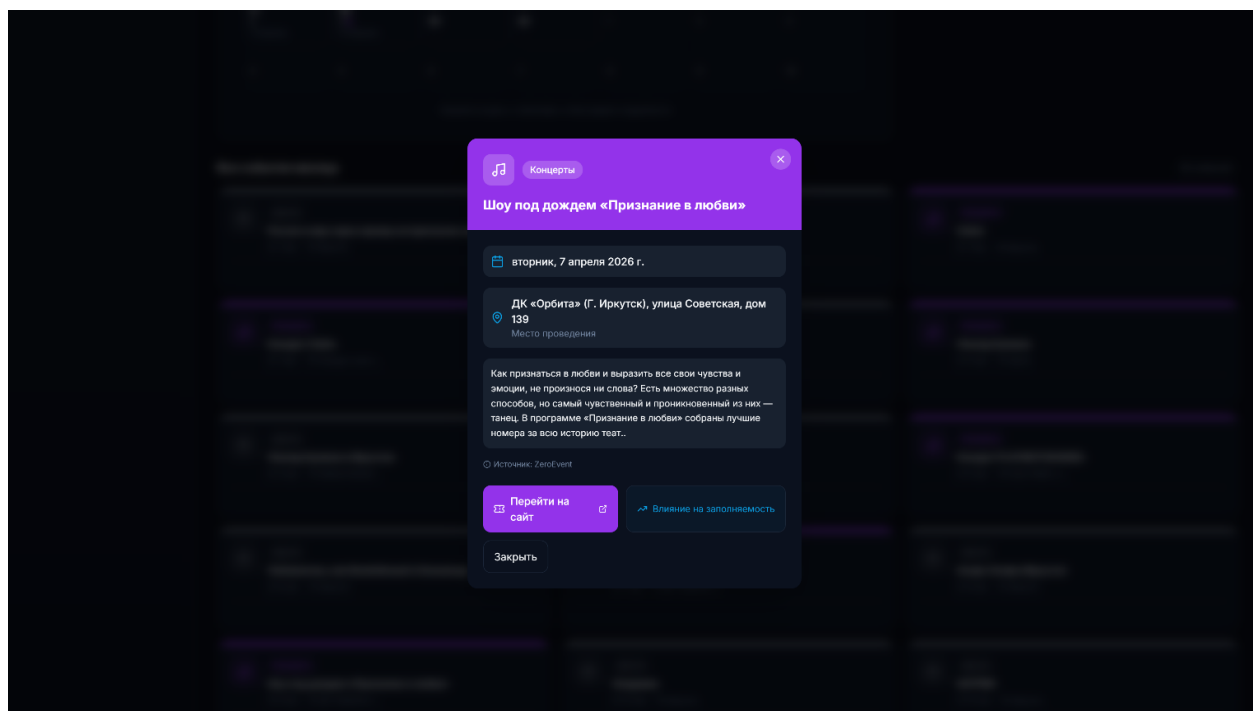
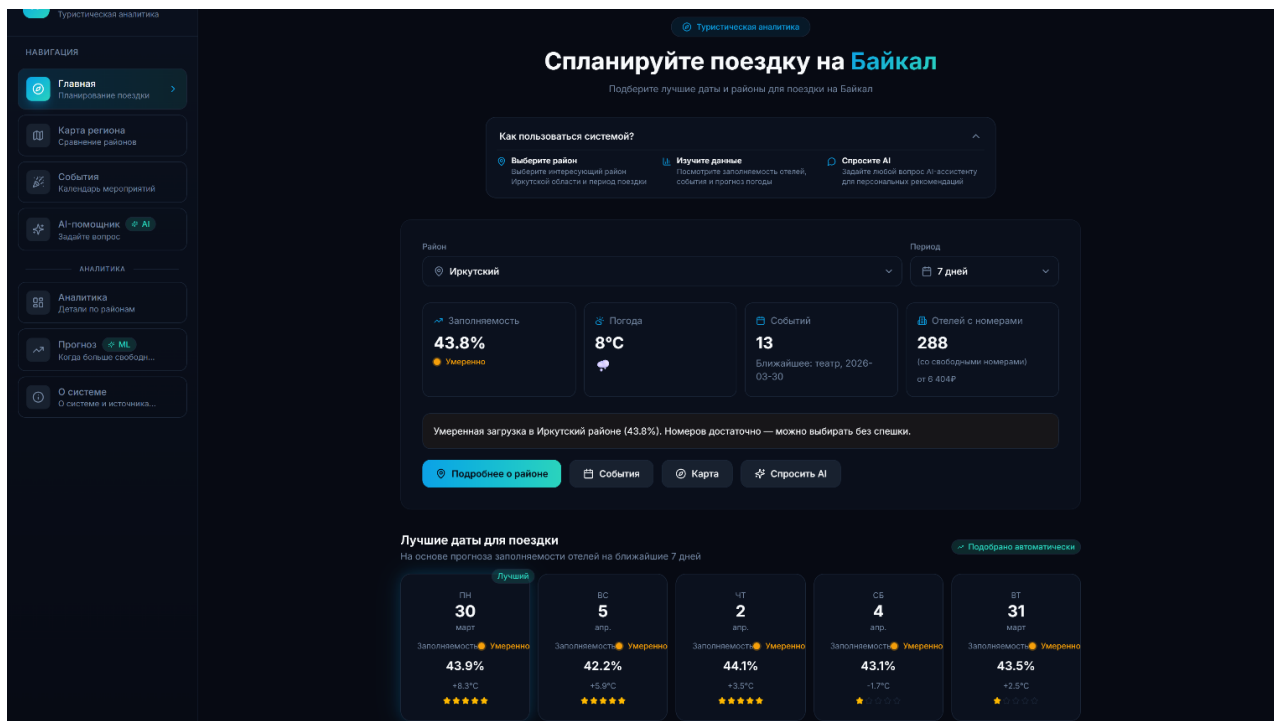
ПРИЛОЖЕНИЯ

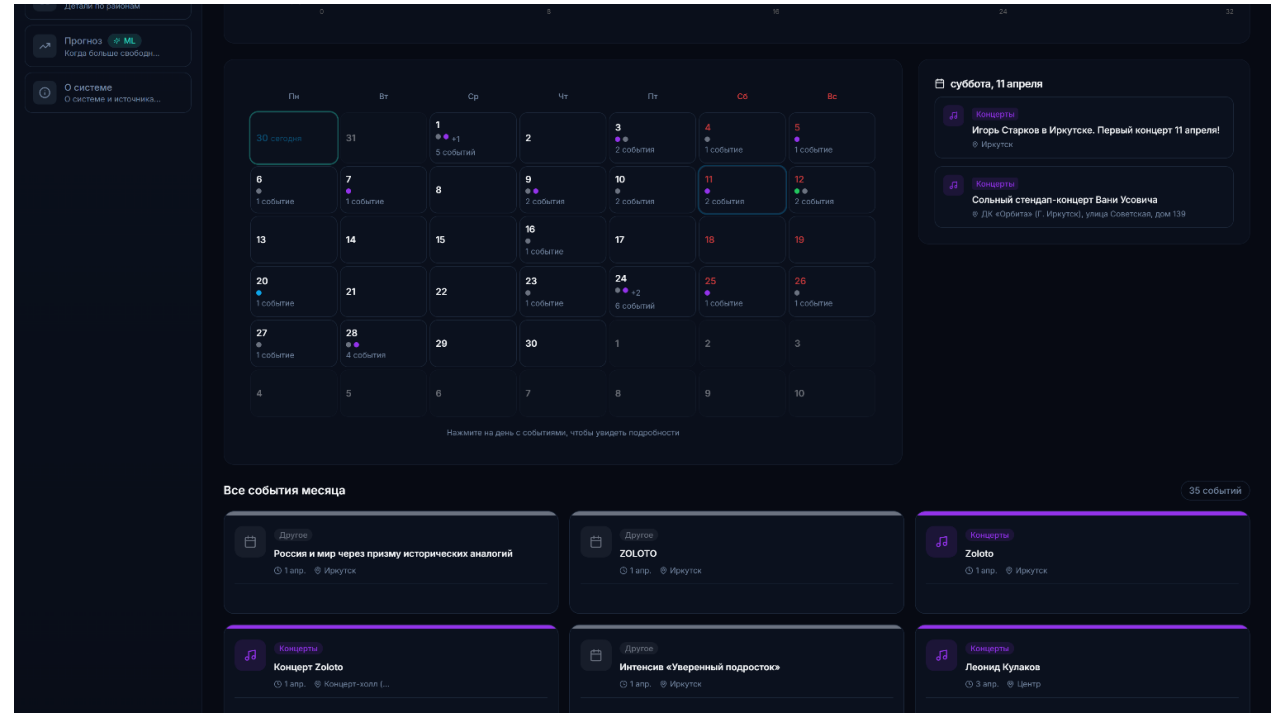
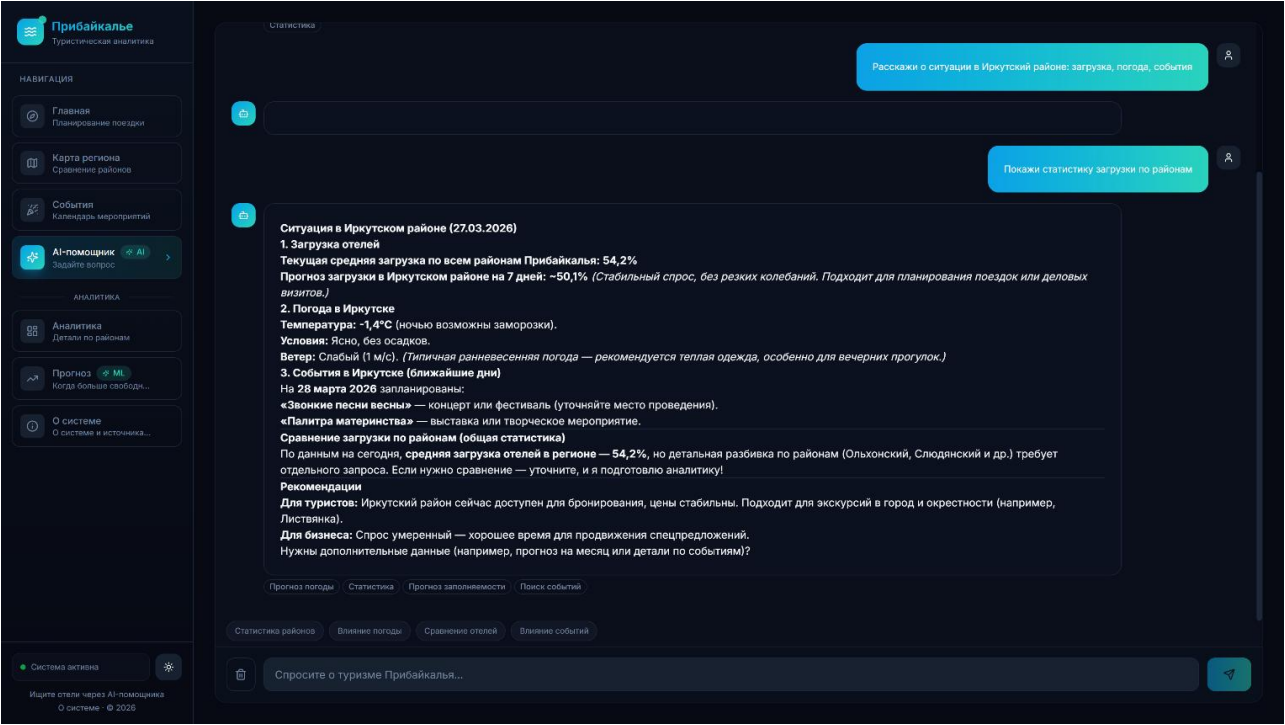
Приложение А

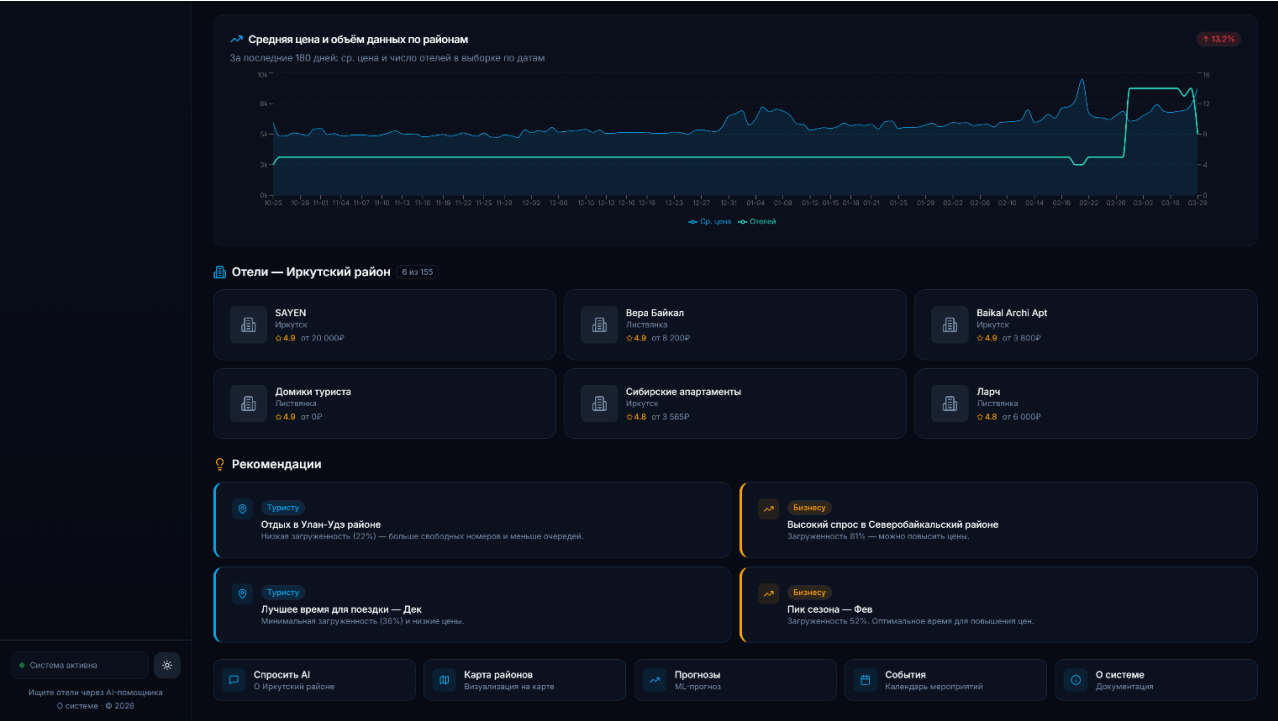
Скриншоты интерфейса системы

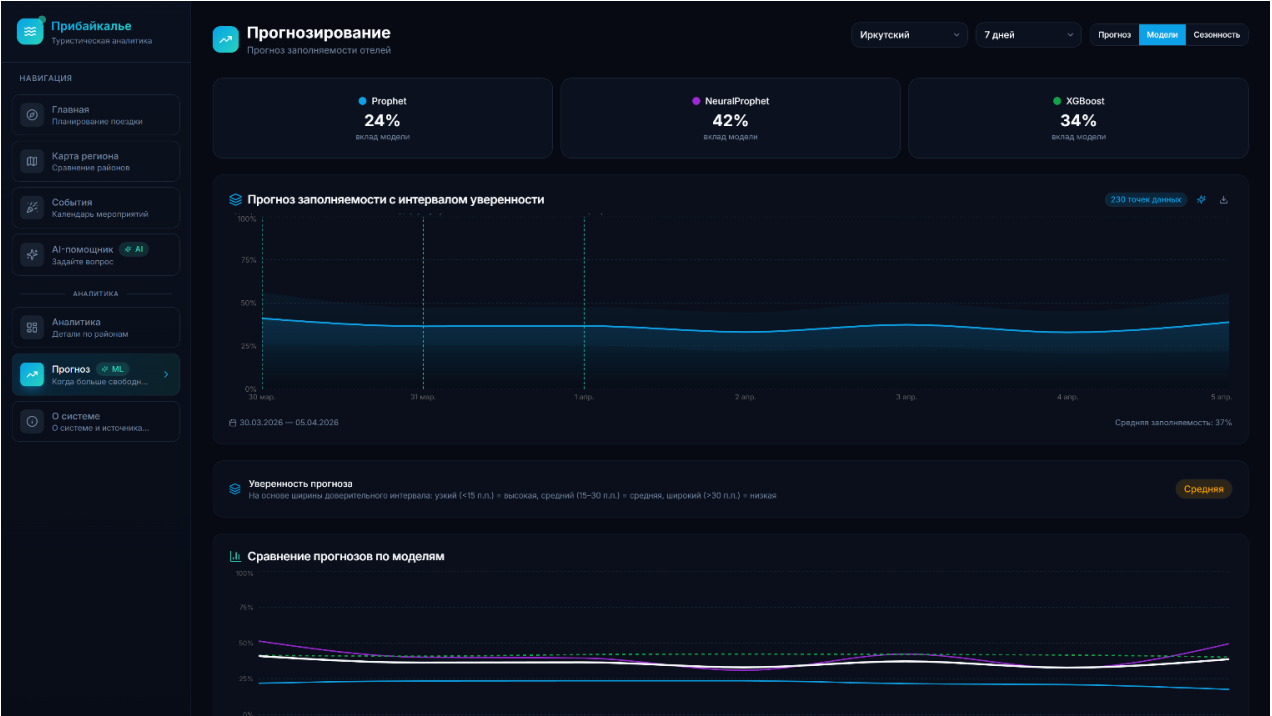
Видеообзор веб-приложения – https://disk.yandex.ru/i/hK8zf_9uJVLQhg

Скриншоты на облачном диске – https://disk.yandex.ru/d/v_GAMtcxMvp3fg











Техническая конфигурация инфраструктуры

Контейнеризация реализована через Docker Compose с двумя режимами: по умолчанию запускаются PostgreSQL 16 и Redis 7 для локальной разработки; профиль `full` добавляет контейнеры серверной и клиентской частей для развёртывания. Секреты передаются через переменные окружения. Контейнеры работают от непривилегированного пользователя (`non-root`) для снижения поверхности атаки. Проверки работоспособности (`healthcheck: pg_isready, redis-cli ping`) обеспечивают мониторинг состояния.

Nginx конфигурация включает: проксирование `/api/*` на серверную часть (`proxy_pass http://backend:8000`), сжатие данных (`gzip`) статических ресурсов, поддержку SSE-соединений (`proxy_buffering off, proxy_read_timeout 300s`), заголовки безопасности `Content-Security-Policy`, `Strict-Transport-Security` (`max-age=31536000`), `X-Content-Type-Options` (`nosniff`).

Безопасность API:

- ограничение частоты запросов: упорядоченное множество Redis (`sorted set`), алгоритм скользящего окна — 60 запросов/мин для общих конечных точек, 10 для `/api/query`, 5 для `/api/parser`; при превышении — HTTP 429 с заголовком повторной попытки (`Retry-After`); при недоступности Redis — резервный механизм в оперативной памяти.

- API-ключ: заголовок `X-API-Key` защищает 12 парсерных маршрутов.

- валидация: `Pydantic Field` (`ge=0, le=100`) для процентов, `Field(min_length=1)` для строк; параметр `days_ahead` принимает значения от 1 до 90; символы `%` и `_` в LIKE-выражениях экранируются для предотвращения SQL-инъекций.

- CORS: ограничение источников запросов, методы `GET`, `POST`, `PUT`, `OPTIONS`, заголовки `Content-Type`, `Authorization`, `X-API-Key`.

Структура API (59 эндпоинтов)

Роутер	Кол-во	Примеры
analytics	17	/api/analytics/kpi, /api/analytics/districts, /api/analytics/hotels-map
parser	12	/api/parser/hotels, /api/parser/events/all
forecast	9	/api/forecast/ensemble, /api/forecast/compare-all
events	6	/api/events, /api/events/stats
query	5	/api/query, /api/query/stream
documents	5	/api/documents/search
hotels	3	/api/hotels, /api/hotels/{id}
root	2	/health, /

Таблица признаков модуля FeatureEngineering

Полный перечень 38 признаков, генерируемых модулем FeatureEngineering

Группа	Признак	Тип	Описание / диапазон значений
Календарные	day_of_week	Int	День недели (0 — пн, 6 — вс)
Календарные	month	Int	Месяц (1–12)
Календарные	quarter	Int	Квартал (1–4)
Календарные	week_of_year	Int	Номер недели в году (1–53)
Календарные	day_of_month	int	День месяца (1–31)
Календарные	is_weekend	Bool	Суббота или воскресенье
Календарные	is_month_start	Bool	Первый день месяца
Календарные	is_month_end	Bool	Последний день месяца
Календарные	is_holiday	Bool	Государственный праздник РФ
Календарные	days_to_holiday	Int	Дней до ближайшего праздника (0–365)
Календарные	days_from_holiday	Int	Дней после последнего праздника (0–365)
Календарные	is_long_weekend	Bool	Выходной или ПН/ПТ при $days_to_holiday \leq 1$
Календарные	season	int	Сезон (0 — зима, 1 — весна, 2 — лето, 3 — осень)
Лаговые	occupancy_lag_1	Float	Заполняемость 1 день назад (%)
Лаговые	occupancy_lag_7	Float	Заполняемость 7 дней назад (%)
Лаговые	occupancy_lag_14	Float	Заполняемость 14 дней назад (%)

Лаговые	occupancy_lag_30	Float	Заполняемость 30 дней назад (%)
Лаговые	occupancy_lag_90	Float	Заполняемость 90 дней назад (%)
Лаговые	occupancy_diff_1	Float	Разность заполняемости за 1 день (п.п.)
Лаговые	occupancy_diff_7	Float	Разность заполняемости за 7 день (п.п.)
Скользящие статистики	rolling_mean_7	Float	Среднее за 7 дней (%)
Скользящие статистики	rolling_mean_30	Float	Среднее за 30 дней (%)
Скользящие статистики	rolling_std_7	Float	Стандартное отклонение за 7 дней
Скользящие статистики	rolling_min_30	Float	Минимум за 30 дней (%)
Скользящие статистики	rolling_max_30	Float	Максимум за 30 дней (%)
Погодные	temperature	Float	Температура воздуха (°C)
Погодные	precipitation	Float	Осадки (мм/сут)
Погодные	temp_deviation	Float	Отклонение от среднемесячной нормы (°C)
Погодные	is_good_weather	Bool	Темп. выше порога и низкие осадки
Событийные	events_count	int	Мероприятий за день (0+)
Событийные	events_week	int	Мероприятий за неделю (0+)
Событийные	has_major_event	bool	Наличие крупного события
Трендовые	trend_index	int	Порядковый индекс даты
Трендовые	normalized_trend	Float	Нормализованный тренд (0–1)
Ценовые	price_current	Float	Текущая мин. стоимость номера (руб.)

Ценовые	price_lag_7	Float	Мин. стоимость 7 дней назад (руб.)
Ценовые	price_change_pct	Float	Изменение цены за неделю (%)
Ценовые	price_rolling_mean_7	Float	Скользящая средняя цены за 7 дней (руб.)

Ежедневные записи студента по практике

Дата	Описание работы, выполненной студентом	Отметка руководителя
24.02	Согласование с руководителем план-графика работы по практике. Изучение задания на практику, уточнение требований к системе прогнозирования заполняемости средств размещения	
25.02 – 26.02	Анализ существующей базы данных парсера 101Hotels, сформированной в рамках курсовой работы. Исследование аналогов систем мониторинга туристической активности (RMS-платформы IDEaS и Duetto, Airbnb Data Dashboard, Yandex DataLens, Google Travel Insights). Формирование требований к разрабатываемой системе	
27.02 – 02.03	Сравнительный анализ серверных фреймворков (FastAPI, Django, Flask) и выбор FastAPI. Исследование СУБД: сравнение PostgreSQL, MySQL, продолжения использования YDB – обоснование миграции на PostgreSQL 16. Выполнение миграции базы данных из облачной YDB (Yandex Database) на локальный PostgreSQL 16. Импорт исторических данных из CSV-файлов курсовой работы (166 файлов, 37 663 записи статистики). Настройка Docker-контейнера с PostgreSQL и Redis	
03.03 – 04.03	Проектирование схемы базы данных (5 таблиц: Hotel, HotelStatistic, Event, QueryHistory, Forecast) в SQLAlchemy ORM. Исследование источников данных о событиях Иркутской области: анализ порталов irk.ru, culture38.ru, zeroevent.ru на наличие структурированных данных	
05.03 – 06.03	Разработка парсеров событий irk.ru/afisha и culture38.ru (aiohttp, BeautifulSoup). Исследование дополнительных источников: kassir.ru, Yandex Afisha, culture.ru, Telegram-каналы. Разработка парсеров zeroevent.ru (JSON-LD) и kassir.ru. Интеграция данных TripAdvisor через API сервиса Xotelo для перекрёстной верификации рейтингов и метаданных отелей	
07.03	Разработка парсеров Telegram-канала @visitirkutskregion, Yandex Afisha и culture.ru. Реализация модуля нормализации, валидации и дедупликации данных из всех источников	
10.03–11.03	Исследование подходов к автоматизации сбора данных: сравнение APScheduler, Celery и cron. Настройка APScheduler с интервалами (события – 6 ч, отели – 2 ч, погода – 3 ч). Разработка модуля статистического анализа загруженности по районам	
12.03–16.03	Анализ влияния событийной активности на заполняемость (расчёт корреляции, сравнение периодов). Исследование векторных баз данных для RAG: сравнение ChromaDB, FAISS, Pinecone, pgvector – выбор ChromaDB. Индексация данных в ChromaDB. Исследование моделей эмбедингов: сравнение GigaChat Embeddings, sentence-transformers, Mistral Embed. Выбор GigaChat Embeddings (оптимизация под русский язык). Тестирование семантического поиска	

	по индексированным данным	
17.03–18.03	Исследование методов прогнозирования временных рядов: сравнительный анализ ARIMA, SARIMA, Holt-Winters, Prophet, NeuralProphet, XGBoost. Обоснование выбора ансамблевого подхода. Подготовка данных и feature engineering (25 типов признаков: календарные, лаговые, скользящие, погодные, событийные)	
19.03–20.03	Реализация моделей Prophet, NeuralProphet и XGBoost. Разработка Ensemble-сервиса: взвешенное среднее по метрикам (RMSE, MAE, R ²), динамическая калибровка весов	
21.03–23.03	Исследование фреймворков AI-агентов: сравнение LangGraph, LangChain AgentExecutor, CrewAI, AutoGen. Реализация LangGraph-агента (StateGraph, 5 инструментов). Исследование LLM-провайдеров: сравнение Mistral, GigaChat, Groq, DeepSeek – выбор Mistral Large. Настройка SSE-стриминга для чат-интерфейса, интеграция MemorySaver для контекста диалога	
24.03–25.03	Исследование фронтенд-стека: сравнение React, Vue, Svelte; Vite vs webpack; Tailwind CSS vs CSS Modules; Recharts vs Chart.js vs D3. Настройка проекта React + TypeScript + Vite 7, подключение TailwindCSS 4. Перенос визуализаций с облачного Yandex DataLens на встроенные графики React (Recharts для аналитики, ECharts для интерактивной карты). Разработка страниц «Аналитика» и «Прогнозы»	
26.03–27.03	Разработка страниц «Карта» (ECharts GeoMap), «События» (каталог с фильтрацией по 8 источникам), «Чат с AI-помощником» (SSE-стриминг). Реализация дизайн-системы: компоненты Button, Card, Badge, Input, Dropdown	
28.03–30.03	Разработка страниц «Планирование поездки» и «О системе». Сквозная интеграция фронтенда и бэкенда, проверка пользовательских сценариев. Глобальный аудит кодовой базы: исправление асинхронных вызовов, обработка ошибок, оптимизация производительности	
31.03	Доработка UX/UI: адаптивность, обработка ошибок, состояния загрузки. Написание unit-тестов и E2E-тестов. Исследование подходов к обеспечению безопасности: rate limiting, API key auth, CSP	
01.04–02.04	Оформление результатов исследований технологий в виде сравнительных таблиц. Документирование разработанных модулей и API (59 эндпоинтов, 7 роутеров)	
03.04	Подготовка отчёта по практике. Оформление ежедневных записей, формирование списка использованных источников	
04.04	Сдача отчета по практике и презентация результатов	

Отзыв руководителя:

Исполатов В.П. своевременно и успешно выполнил задачи, поставленные в рамках производственной практики, проявил самостоятельность и достаточный уровень знаний и навыков в области

прикладной информатики.

Студент осуществил анализ и выявил недостатки существующих систем для решения задачи анализа заполненности средств размещения, выполнил анализ данных о средствах размещения, туристских мероприятия/событиях и метеорологических условиях территории, обосновал выбор инструментов для сбора, хранения, обработки и визуализации данных, выполнил проектирование архитектуры приложения, логической модели базы данных (БД), разработал метод прогнозирования заполняемости средств размещения на основе моделей машинного обучения. В рамках практики реализованы БД, методы сбора и прогнозирования данных, интеллектуальный агент и пользовательский интерфейс, выполнил оценку модели машинного обучения. В качестве замечаний необходимо отметить, что представленные результаты проектирования получены на основе использования неизвестных методологий и нотаций. Считаю, что Исполатов В.П. заслуживает положительной оценки за выполненную работу.

04 апреля 2026

(дата)

О.А. Николайчук

(И.О. Фамилия
руководителя задачи)

(подпись
руководителя задачи)